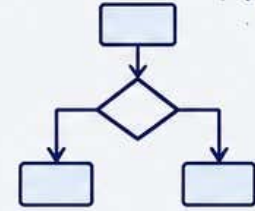
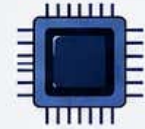


1010101010
0101010101
1010101010
0101010101



Infographics



Visual Learning

Better understanding through visual content.



Simplified Concepts

Complex topics explained in simple and visual form.



Quick Revision

Easy to revise important points and diagrams.

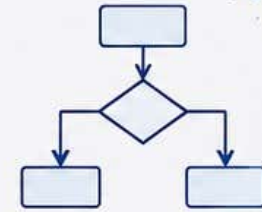


Exam Ready

Helps in faster revision and better exam preparation.



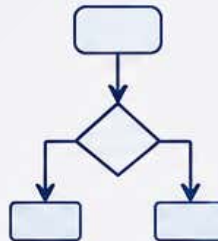
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101



INTRODUCTION TO JAVA : HISTORY AND EVOLUTION

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems. It is designed to have as few implementation dependencies as possible.

"Write Once, Run Anywhere"

1. HISTORY OF JAVA

- Java was developed by James Gosling and his team at Sun Microsystems (now a subsidiary of Oracle).
- The project was started in 1991 under the name "Oak".
- Initially designed for interactive television, set-top boxes and other consumer electronic devices.
- In 1995, Oak was renamed "Java", inspired by Java coffee, which is known for its richness and energy.
- Sun Microsystems officially released Java in 1996.
- In 2009, Oracle Corporation acquired Sun Microsystems and now manages and maintains Java.

2. KEY FEATURES OF JAVA (AT INTRODUCTION)

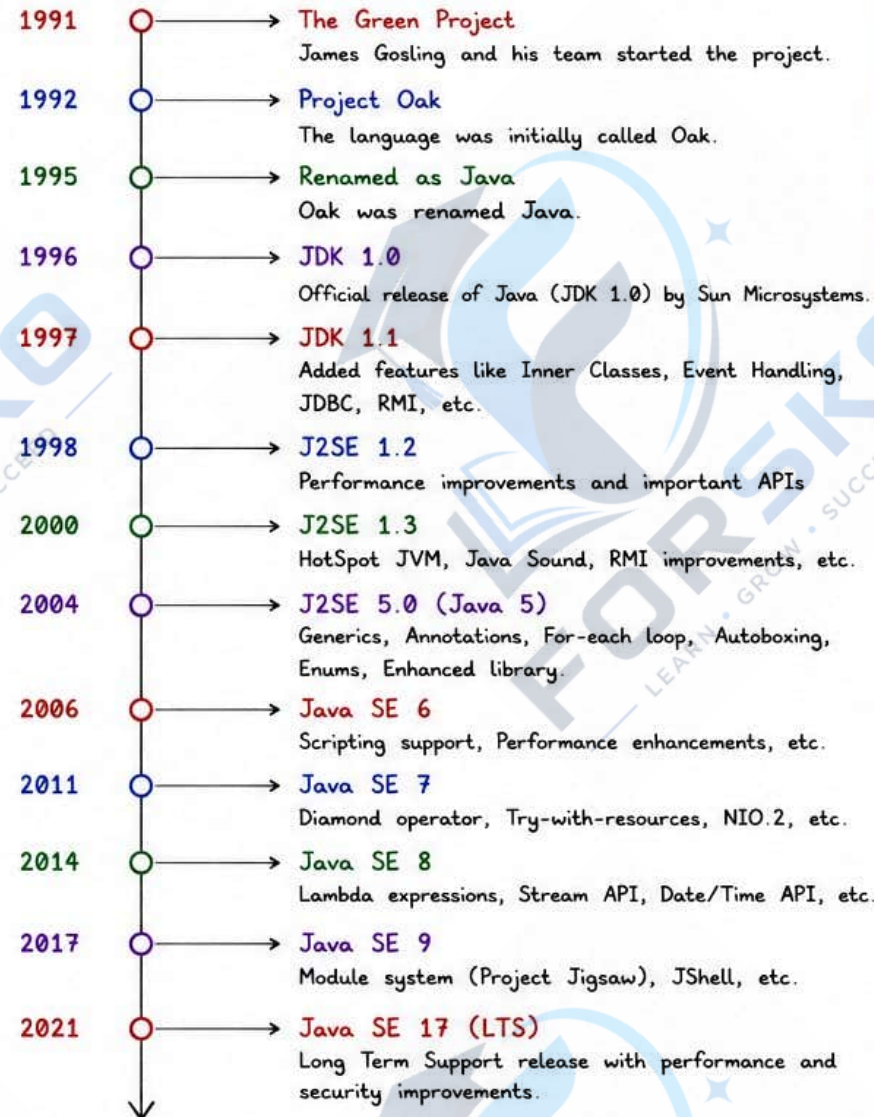
- ★ Simple
- ★ Object-Oriented
- ★ Platform Independent
(Write Once, Run Anywhere)
- ★ Secure
- ★ Robust
- ★ Architecture Neutral
- ★ Portable
- ★ High Performance
- ★ Multithreaded
- ★ Distributed
- ★ Interpreted



NOTE :

Java was designed to be reliable, secure and portable so that it can be used to develop applications for a wide range of platforms and devices.

3. EVOLUTION OF JAVA



KEY POINTS

- ✓ Java has come a long way from a small project "Oak" to a powerful, widely-used programming language.
- ✓ It has evolved continuously with new features and improvements.
- ✓ Java is now used in web, mobile, enterprise applications, Android, cloud, big data, AI/ML and more.

JAVA : FEATURES

Java is a powerful, general-purpose, object-oriented programming language. It provides many features that make it simple, secure, platform-independent and widely used.

KEY POINTS

- ★ Java is simple, object-oriented and secure.
- ★ It is platform independent and portable.
- ★ It provides high performance and multithreading.
- ★ It is robust, distributed and has dynamic nature.
- ★ Java supports automatic memory management.

MAJOR FEATURES OF JAVA

1 Simple



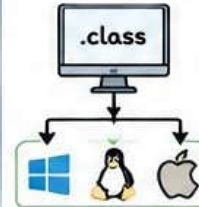
- Java has a simple syntax similar to C/C++.
- It removes complex features like pointers, operator overloading, etc.

2 Object-Oriented



- Java is based on OOPs concepts.
- It supports classes, objects, inheritance, polymorphism, encapsulation and abstraction.

3 Platform Independent



- Java code is compiled to bytecode (.class file)
- Bytecode runs on any platform using JVM.
- Write Once, Run Anywhere.

4 Portable



- Java supports different platforms (Windows, Linux, macOS, etc.).
- It has fixed size data types, which makes it portable.

5 Secure



- Java provides built-in security features.
- No explicit pointer usage, no access to memory addresses.
- It supports authentication and encryption.

6 Robust



- Strong memory management with garbage collection.
- Exception handling and type checking make Java robust.

7 High Performance



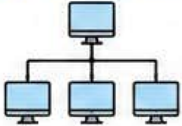
- Java uses Just-In-Time (JIT) compiler.
- Converts bytecode to native code at runtime for better performance.

8 Multithreaded



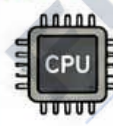
- Java supports multithreading.
- Multiple threads can run concurrently.
- Improves performance of applications.

9 Distributed



- Java is designed for distributed computing.
- It supports APIs like RMI, Socket programming, etc.

10 Architecture Neutral



- Java compiler generates architecture-neutral bytecode.
- Bytecode runs on any processor with JVM.

11 Interpreted



- Java bytecode is interpreted by JVM line by line.
- Easy to debug and test.

12 Dynamic



- Java is dynamic.
- It can adapt to an evolving environment.
- Classes are loaded at runtime.

13 Automatic Memory Management



- Java has Garbage Collection.
- It automatically allocates and de-allocates memory.
- No manual memory management.

14 Strongly Typed



- Java is strongly typed.
- All variables must be declared before use.
- Ensures type safety.

15 High Reliability



- Java provides reliable features like exception handling, garbage collection, etc.
- Reduces system failure.

16 Rich API



- Java provides a rich set of libraries (APIs).
- Helps in rapid development.
- Example: Collections, I/O, Networking, etc.

NOTE



These features make Java a powerful, flexible, secure and widely used programming language for developing desktop, web, mobile and enterprise applications.

In Short

Simple • Object-Oriented • Platform Independent • Portable • Secure • Robust
High Performance • Multithreaded • Distributed • Dynamic • Rich API and more.

JRE (Java Runtime Environment)

JRE (Java Runtime Environment) is a part of Java Software that provides libraries, Java Virtual Machine (JVM) and other components required to **run** Java programs.

KEY POINTS

- ✓ JRE is required to **run** Java applications.
- ✓ It does not contain tools for development.
- ✓ It includes JVM, class libraries and other supporting files.
- ✓ It is platform dependent.
- ✓ It is a part of JDK (Java Development Kit).

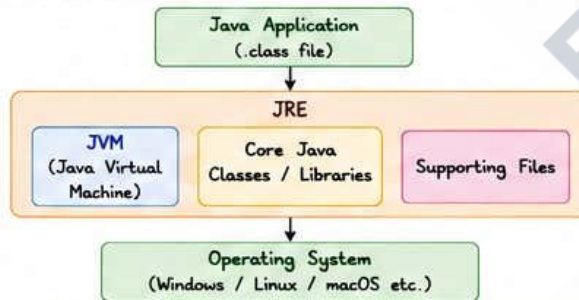
1. WHAT IS JRE?

- JRE provides the environment in which Java bytecode can be executed.
- It includes JVM, core class libraries and other supporting files.
- It is used to **run** Java applications on any platform.

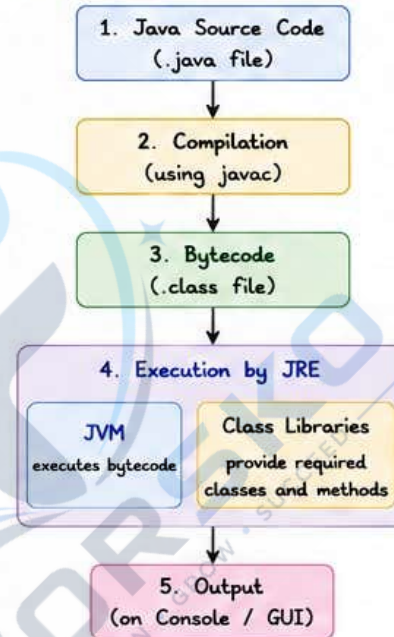
2. COMPONENTS OF JRE

- **JVM (Java Virtual Machine)**
Executes the Java bytecode.
- **Core Java Classes / Libraries**
A set of standard class libraries required by Java programs.
- **Supporting Files**
Resource files and configurations required by JRE.

3. JRE ARCHITECTURE



4. HOW JRE WORKS?



1. Java Source Code

Programmer writes code in .java file.

2. Compilation

The source code is compiled using javac compiler to generate bytecode (.class file).

3. Bytecode

Bytecode is platform independent code generated after compilation.

4. Execution by JRE

- JVM loads and verifies the bytecode.
- JVM executes the bytecode.
- Class libraries provide the required classes and methods to the program.

5. Output

The result is displayed on the console or in the GUI.

5. JRE vs JDK

Feature	JRE	JDK
Full Form	Java Runtime Environment	Java Development Kit
Purpose	To run Java programs	To develop and run Java programs
Contains	JVM, Class Libraries, Supporting Files	JRE + Development Tools (javac, java, jdb, javadoc, etc.)
Used By	End Users	Developers
Installation Size	Smaller	Larger
Example	To run .class file	To compile and run .java file

6. BENEFITS OF JRE

- ★ Enables platform independence.
- ★ Ensures reliability and security.
- ★ Automatic memory management through Garbage Collection.
- ★ Provides rich set of libraries.
- ★ Makes Java programs portable and easy to run.

8. JRE FOLDER STRUCTURE (Example)

jre/
├ bin/ → Contains executables (java, etc.)
├ lib/ → Core libraries (rt.jar, etc.)
├ conf/ → Configuration files
├ legal/ → License and legal information
└ other supporting files and folders

9. IMPORTANT NOTE



- JRE is required only to run compiled Java programs.
- JRE does not provide any tool to write or compile programs.
- For development and execution, JDK is required.

7. EXAMPLE

To run a Java program:

1. Write Program → Hello.java
2. Compile → javac Hello.java
3. Run → java Hello

- When the command "java Hello" is executed, JRE is used to run the program.

Output:

Hello, Welcome to Java!

JDK (Java Development Kit) and JVM (Java Virtual Machine)

JDK is a software development kit used to develop, compile, debug and run Java programs.

JVM is the runtime environment that executes Java bytecode and provides platform independence.

KEY POINTS

- ✓ JDK is used for development.
- ✓ JVM is used for execution.
- ✓ JDK includes JRE and development tools.
- ✓ JVM executes bytecode generated by the compiler.
- ✓ JVM makes Java platform independent.

1. JDK (Java Development Kit)

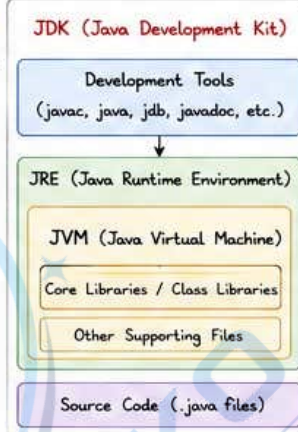
1.1 What is JDK?

- JDK is a complete development environment for Java.
- It is used by developers to create, build, test and run Java applications.
- It includes tools, libraries, JRE and JVM.

1.2 Components of JDK

- JRE (Java Runtime Environment)**
Provides libraries, JVM and other files required to run Java programs.
- JVM (Java Virtual Machine)**
Executes Java bytecode.
- Development Tools**
Tools used to develop and debug Java programs.
- Libraries**
Collection of useful classes and interfaces.

1.3 Structure of JDK



1.4 Tools Provided by JDK

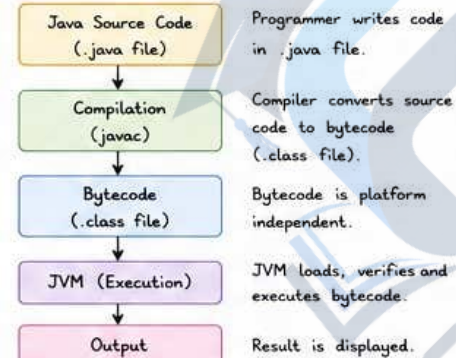
- | | |
|---|--|
| • javac - Java compiler (compiles .java to .class) | • jar - Creates Java archive (.jar) files |
| • java - Runs the Java program | • javap - Disassembles class files |
| • jdb - Java debugger | • jconsole - Monitoring and management tool |
| • javadoc - Generates documentation | • jdeps - Analyzes dependencies |

2. JVM (Java Virtual Machine)

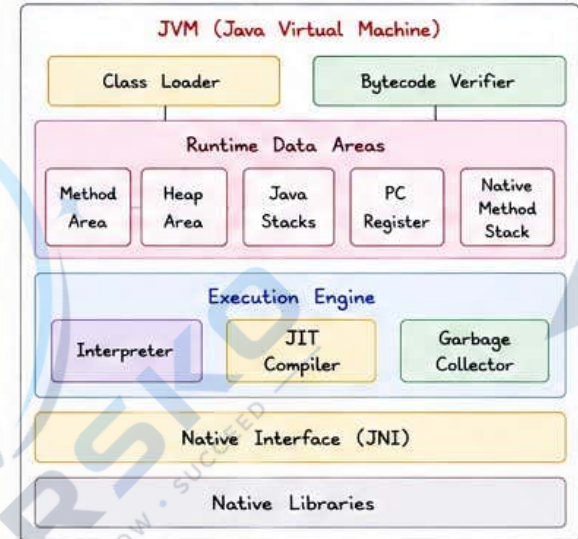
2.1 What is JVM?

- JVM is a runtime engine that executes Java bytecode.
- It provides platform independence.
- It is a part of JRE.
- One JVM can run on many operating systems.

2.2 How JVM Works?



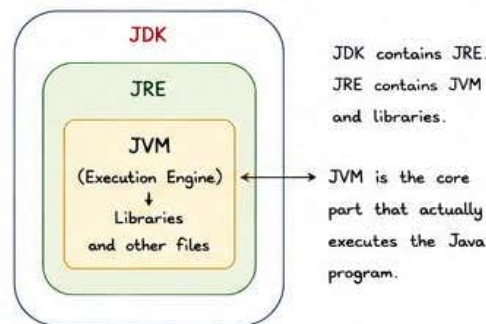
2.3 Architecture of JVM



3. JDK vs JVM

Feature	JDK	JVM
Full Form	Java Development Kit	Java Virtual Machine
Purpose	Used for development of Java applications	Used for execution of Java applications
Contains	JRE, JVM, tools (javac, java, jdb, etc.), libraries	Class loader, verifier, runtime data areas, execution engine, etc.
Used By	Developers	End users
Input	.java source file	.class bytecode
Output	.class file (after compilation) and also runs program	Executes bytecode and produces output
Dependency	Platform dependent (installation)	Platform independent (bytecode level)

4. Relationship



5. Example

Program: Hello.java

```
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Steps:

- Write program in Hello.java
- Compile using: `javac Hello.java` → Hello.class
- Run using: `java Hello`
- JVM executes Hello.class and displays output.

Output:

Hello, Java!



Important Note

- Always use JDK for development.
- Install JDK to get both development tools and runtime environment.

- JVM is available on different platforms, so the same .class file can run anywhere.
- "Write Once, Run Anywhere" is possible because of JVM.

DATA TYPES IN JAVA

- ✓ In Java, data types specify the type of data a variable can hold.
- ✓ Java data types are divided into two categories:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

NOTE

Java is a strongly typed language, so every variable must have a data type.

1. PRIMITIVE DATA TYPES (Built-in / Basic)

Data Type	Size	Default Value	Description	Example
byte	1 byte	0	Stores whole numbers from -128 to 127	<code>byte b = 10;</code>
short	2 bytes	0	Stores whole numbers from -32,768 to 32,767	<code>short s = 200;</code>
int	4 bytes	0	Stores whole numbers (larger range)	<code>int i = 1000;</code>
long	8 bytes	0L	Stores very large whole numbers	<code>long l = 123456789L;</code>
float	4 bytes	0.0f	Single precision floating point number	<code>float f = 5.6f;</code>
double	8 bytes	0.0d	Double precision floating point number	<code>double d = 3.14159;</code>
char	2 bytes	'\u0000'	Stores a single character	<code>char ch = 'A';</code>
boolean	1 bit	false	Stores true or false values	<code>boolean b = true;</code>

2. NON-PRIMITIVE (REFERENCE) DATA TYPES

Type	Description
String	Represents a sequence of characters.
Array	Stores multiple values of same type.
Class	User-defined data type (Objects).
Interface	Collection of abstract methods.

3. TYPE CONVERSION

> Widening (Implicit) :

Smaller type → Larger type

Example: `int i = 10;`
`double d = i; // OK`

> Narrowing (Explicit) :

Larger type → Smaller type

Example: `double d = 10.5;`
`int i = (int)d; // Type casting`



IMPORTANT

- ✓ Primitive types store actual values.
- ✓ Non-primitive types store references (addresses).



KEYWORDS IN JAVA

- ★ Keywords are reserved words in Java that have special meaning. They cannot be used as variable names, method names or class names.

CATEGORIES OF KEYWORDS

1. Access Modifiers

- public
- private
- protected
- default (no keyword)

2. Class & Object Related

- class
- extends
- interface
- this
- object
- implements
- new
- super

3. Control Flow

- if
- case
- while
- continue
- else
- default
- do
- return
- switch
- for
- break

4. Data Types

- boolean
- short
- long
- double
- byte
- int
- float
- char

5. Exception Handling

- try
- finally
- throws
- catch
- throw

6. Other Keywords

- static
- abstract
- synchronized
- transient
- final
- native
- volatile
- strictfp

COMMONLY USED JAVA KEYWORDS (Alphabetical Order)

abstract	assert	boolean	break	byte	case	catch	char
class	const	continue	default	do	double	else	enum
extends	final	finally	float	for	goto*	if	implements
import	instancetof	int	interface	long	native	new	package
private	protected	public	return	short	static	strictfp	super
switch	synchronized	this	throw	throws	transient	try	void
volatile	while	true	false	null			

EXAMPLES

- ✓ Valid variable name : `int age;`
- ✗ Invalid variable name : `int class;` // class is keyword
- ✓ Valid method name : `void showData() {}`
- ✗ Invalid method name : `void if() {}` // if is keyword



TIPS

- ✓ Keywords are all lowercase.
- ✓ true, false and null are also special keywords.

JAVA OPERATORS

Operators are special symbols that are used to perform operations on variables and values.

Java is rich in built-in operators.

1. TYPES OF OPERATORS IN JAVA

- Arithmetic Operators
- Relational (Comparison) Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Unary Operators
- Ternary (Conditional) Operator
- Type Comparison Operator

2. ARITHMETIC OPERATORS

Operator	Meaning	Example	Result
+	Addition	int a = 10 + 5;	15
-	Subtraction	int a = 10 - 5;	5
*	Multiplication	int a = 10 * 5;	50
/	Division	int a = 10 / 5;	2
%	Modulus (Remainder)	int a = 10 % 3;	1

Note: If either operand is double, the result is double.

3. RELATIONAL (COMPARISON) OPERATORS

Operator	Meaning	Example	Result
==	Equal to	10 == 10	true
!=	Not equal to	10 != 20	true
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	10 >= 10	true
<=	Less than or equal to	10 <= 5	false

4. LOGICAL OPERATORS

Operator	Meaning	Example	Result
&&	Logical AND	(10 > 5) && (20 > 10)	true
	Logical OR	(10 > 5) (20 < 10)	true
!	Logical NOT	!(10 > 5)	false

Note: Logical operators work on boolean values (true/false).

5. BITWISE OPERATORS

Operator	Meaning	Example	Result
&	Bitwise AND	5 & 3 (0101 & 0011)	1 (0001)
	Bitwise OR	5 3 (0101 0011)	7 (0111)
^	Bitwise XOR	5 ^ 3 (0101 ^ 0011)	6 (0110)
~	Bitwise Complement	~5 (~0000 0101)	-6
<<	Left Shift	5 << 1	10 (1010)
>>	Right Shift	5 >> 1	2 (0010)
>>>	Unsigned Right Shift	5 >>> 1	2 (0010)

Note: Bitwise operators work on integral types.

6. ASSIGNMENT OPERATORS

Operator	Example	Equivalent To
=	a = 10	a = 10
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5
&=	a &= 5	a = a & 5
=	a = 5	a = a 5
^=	a ^= 5	a = a ^ 5
<<=	a <<= 2	a = a << 2
>>=	a >>= 2	a = a >> 2
>>>=	a >>>= 2	a = a >>> 2

9. OPERATOR PRECEDENCE

Precedence	Operator	Description
1	()	Parentheses
2	++ -- + - ! ~	Unary
3	* / %	Multiplicative
4	+ -	Additive
5	<< >> >>>	Shift
6	< <= > >=	Relational
7	== !=	Equality
8	&	Bitwise AND
9	^	Bitwise XOR
10		Bitwise OR
11	&&	Logical AND
12		Logical OR
13	?:	Ternary
14	= += -= *= /= %= &= = ^= <<= >>= >>>=	Assignment

Note: Higher precedence operator is evaluated first.

7. UNARY OPERATORS

Operator	Meaning	Example	Result
++	Increment (increase by 1)	int a = 5; a++;	a = 6
--	Decrement (decrease by 1)	int a = 5; a--;	a = 4
+	Unary Plus (no change)	int a = 5; +a;	a = 5
-	Unary Minus (change sign)	int a = 5; -a;	a = -5
!	Logical NOT (invert)	boolean b = true; !b;	false
~	Bitwise Complement	int a = 5; ~a;	-6

8. TERNARY (CONDITIONAL) OPERATOR

Condition ? expression1 : expression2
If condition is true, expression1 is executed
otherwise expression2 is executed.

Example:

```
int max = (a > b) ? a : b;
```

Equivalent to:

```
int max;  
if(a > b)  
    max = a;  
else  
    max = b;
```

10. TYPE COMPARISON OPERATOR

instanceof

Used to test whether an object is an instance of a class, subclass or interface.

Example:

```
String s = "Hello";  
System.out.println(s instanceof String); // true  
System.out.println(s instanceof Object); // true
```



IMPORTANT POINTS

- Operators are used with operands (variables, values, expressions).
- The result of an operation depends on the type of operands.
- In case of different data types, Java automatically promotes smaller type to larger type.

EXAMPLES

```
int a = 10, b = 3;  
a + b = 13  
a % b = 1  
a > b && a < 20 // true
```

```
int x = 5;  
x++ ; // 6  
++x ; // 7  
x > 5 ? "Yes" : "No" // Yes
```

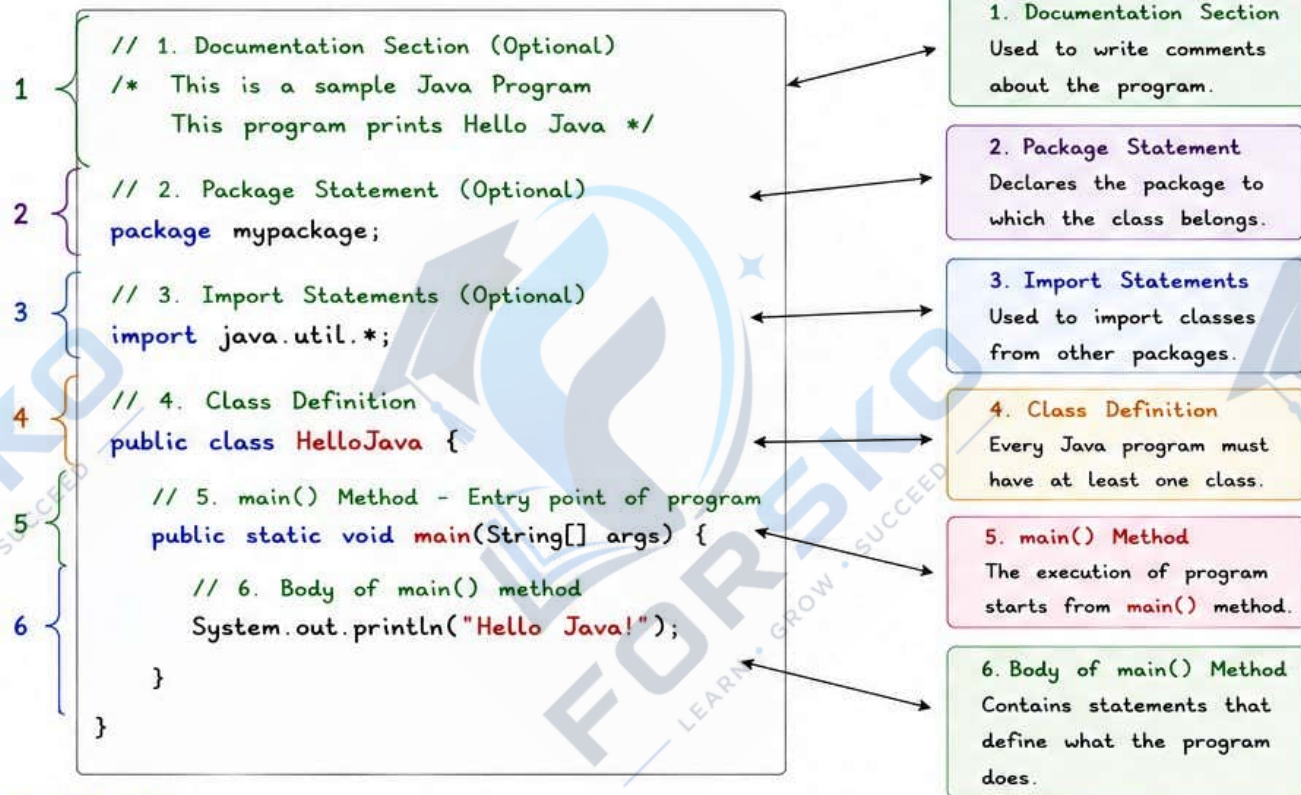
NOTE

Knowing operators and their precedence is very important for writing correct and efficient Java programs.

STRUCTURE OF JAVA PROGRAM

A Java program is a collection of classes. Every Java program must contain a class with **main()** method. The execution of program starts from the **main()** method.

1. GENERAL FORM OF JAVA PROGRAM



4. EXAMPLE

```
Program
package mypackage;
import java.util.*;
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello Java!");
    }
}
```

Output

Hello Java!

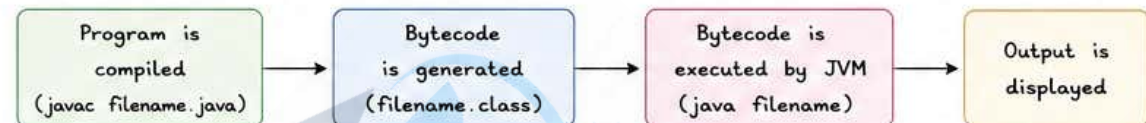
2. EXPLANATION OF PARTS

Part	Description
Documentation Section	Comments that describe the program.
Package Statement	(Optional) Specifies the package name using package keyword.
Import Statements	(Optional) Imports classes/interfaces from other packages using import keyword.
Class Definition	Defines a class using class keyword. The file name should be same as the public class name.
main() Method	<code>public static void main(String[] args)</code> is the entry point of every Java program.
Body of main() Method	Contains the executable statements of the program.

3. POINTS TO REMEMBER

- ✓ Every Java program must have main() method.
- ✓ Class name and file name must be same (if class is public).
- ✓ Java is case-sensitive.
- ✓ Statements are executed in sequence inside main() method.

5. FLOW OF EXECUTION



JAVA COMMAND LINE ARGUMENTS

Command line arguments are the values passed to a Java program at the time of running from the command prompt or terminal. These arguments are received by the `main()` method.

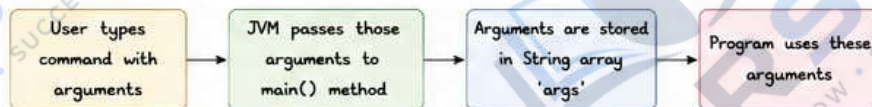
1. What are Command Line Arguments?

- When we run a Java program, we can pass some values along with the program name.
- These values are called command line arguments.
- In Java, command line arguments are received in the `String` array passed to `main()` method.

2. Syntax

```
public class ClassName {  
    public static void main(String[] args) {  
        // args is an array that stores command line arguments  
    }  
}
```

3. How it Works?



4. Example Program

```
public class CommandLineDemo {  
    public static void main(String[] args) {  
        System.out.println("Total arguments: " + args.length);  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("args[" + i + "] = " + args[i]);  
        }  
    }  
}
```

9. Example: Add Two Numbers Using Command Line Arguments

```
public class CommandLineDemo {  
    public static void main(String[] args) {  
        if (args.length != 2) {  
            System.out.println("Please provide two numbers");  
            return;  
        }  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        int sum = a + b;  
        System.out.println("Sum = " + sum);  
    }  
}
```

Run:

```
javac AddTwoNumbers.java
```

```
java AddTwoNumbers 25 37
```

Output:

```
Sum = 62
```

If wrong arguments:

```
java AddTwoNumbers 25
```

Output:

```
Please provide two numbers
```

5. Running the Program

Compile the program

```
javac CommandLineDemo.java
```

Run the program with arguments

```
java CommandLineDemo 10 Hello Java Programming
```

6. Output

```
Total arguments: 3  
args[0] = 10  
args[1] = Hello  
args[2] = Java  
args[3] = Programming
```



Note

- The first argument is at index 0.
- If no arguments are passed, `args.length` will be 0.
- All arguments are treated as `String` in Java.

7. Accessing Arguments

Expression	Description
<code>args.length</code>	Returns total number of arguments
<code>args[i]</code>	Returns argument at index i
<code>args[0]</code>	First argument
<code>args[args.length - 1]</code>	Last argument

8. Points to Remember

- ✓ `main()` must have `String[] args` to receive command line arguments.
- ✓ Arguments are separated by spaces while running.
- ✓ Order of arguments is preserved.
- ✓ Arguments are always passed as Strings.

SIMPLE JAVA PROGRAMS

Here are some simple Java programs for beginners.

1. Print "Hello, World!"

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Output:
Hello, World!

2. Add Two Numbers

```
public class AddTwoNumbers {  
    public static void main(String[] args) {  
        int a = 10, b = 20, sum;  
        sum = a + b;  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output:
Sum = 30

3. Find the Largest Number

```
public class LargestNumber {  
    public static void main(String[] args) {  
        int a = 25, b = 40, c = 15, largest;  
        largest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);  
        System.out.println("Largest = " + largest);  
    }  
}
```

Output:
Largest = 40

4. Check Even or Odd

```
public class EvenOdd {  
    public static void main(String[] args) {  
        int num = 7;  
        if (num % 2 == 0)  
            System.out.println(num + " is Even");  
        else  
            System.out.println(num + " is Odd");  
    }  
}
```

Output:
7 is Odd

5. Factorial of a Number

```
public class Factorial {  
    public static void main(String[] args) {  
        int n = 5, fact = 1;  
        for (int i = 1; i <= n; i++)  
            fact = fact * i;  
        System.out.println("Factorial of " + n + " = " + fact);  
    }  
}
```

Output:
Factorial of 5 = 120

6. Table of a Number

```
public class Table {  
    public static void main(String[] args) {  
        int n = 5;  
        for (int i = 1; i <= 10; i++)  
            System.out.println(n + " x " + i + " = " + (n * i));  
    }  
}
```

Output:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50

7. Sum of First N Natural Numbers

```
public class SumNatural {  
    public static void main(String[] args) {  
        int n = 10, sum = 0;  
        for (int i = 1; i <= n; i++)  
            sum = sum + i;  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output:
Sum = 55

8. Reverse a Number

```
public class ReverseNumber {  
    public static void main(String[] args) {  
        int num = 12345, rev = 0, rem;  
        while (num != 0) {  
            rem = num % 10;  
            rev = rev * 10 + rem;  
            num = num / 10;  
        }  
        System.out.println("Reverse = " + rev);  
    }  
}
```

Output:
Reverse = 54321

9. Check Prime Number

```
public class PrimeNumber {  
    public static void main(String[] args) {  
        int num = 17, i, flag = 0;  
        if (num <= 1)  
            flag = 1;  
        else {  
            for (i = 2; i <= num/2; i++) {  
                if (num % i == 0) { flag = 1; break; }  
            }  
        }  
        if (flag == 0)  
            System.out.println(num + " is Prime");  
        else  
            System.out.println(num + " is Not Prime");  
    }  
}
```

Output:
17 is Prime

10. Simple Interest

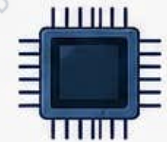
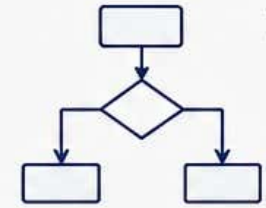
```
public class SimpleInterest {  
    public static void main(String[] args) {  
        double p = 1000, r = 5.5, t = 2, si;  
        si = (p * r * t) / 100;  
        System.out.println("Simple Interest = " + si);  
    }  
}
```

Output:
Simple Interest = 110.0

Note:

- Java is case-sensitive.
- Class name should be same as file name.
- Use meaningful variable names.

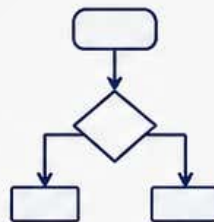
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



Decision Making and Branching: Simple if statement

1. Introduction



The **if** statement is used to execute a block of code only when a specified condition is true.

★ It is the simplest form of decision making in Java.

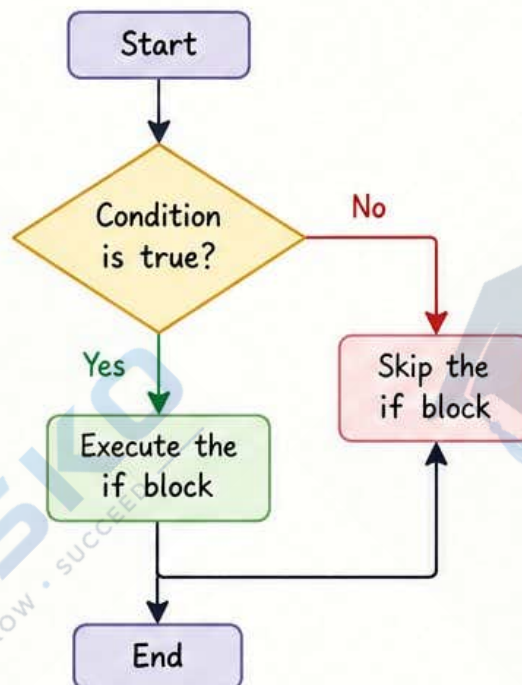
3. Syntax

```
if (condition) {  
    // statements to be executed  
}
```

Example Code

```
int age = 18;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

2. Flowchart



4. Example

```
public class IfExample {  
    public static void main(String[] args) {  
        int number = 10;  
        if (number > 0) {  
            System.out.println("Number is positive");  
        }  
    }  
}
```

Output:
Number is positive

5. Key Points

- ➔ If the condition is **true**, the block inside **if** is executed.
- ➔ If the condition is **false**, the block is skipped.
- ➔ **If** has no **else** part.

6. Notes

- The condition must be a boolean expression.
- Use simple **if** when you want to execute a block only in one case.



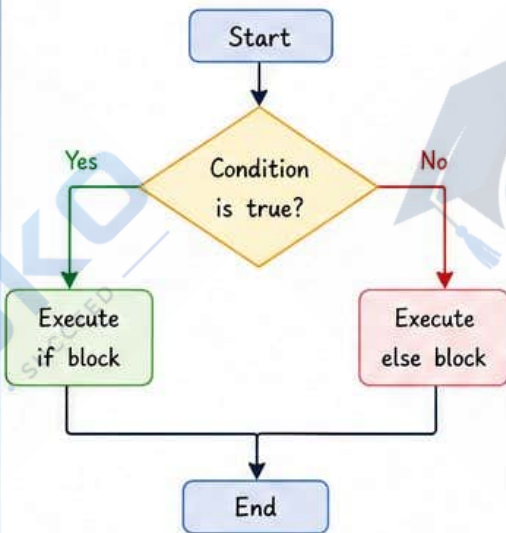
★ Simple if statement is the building block for more complex decision making like **if-else**, **else-if ladder** and **nested if**.

Decision Making and Branching in Java

1. if...else statement

The **if...else** statement is used to execute one block of code if a condition is **true** and another block if the condition is **false**.

Flowchart:



Output:

Number is negative

Key Points:

- If the condition is **true**, the **if** block is executed.
- If the condition is **false**, the **else** block is executed.
- Only one block (**if** or **else**) is executed.

Syntax:

```
if (condition) {  
    // statements when condition is true  
}  
else {  
    // statements when condition is false  
}
```

Example:

```
public class IfElseExample {  
    public static void main(String[] args) {  
        int num = -5;  
        if (num >= 0) {  
            System.out.println("Number is positive");  
        }  
        else {  
            System.out.println("Number is negative");  
        }  
    }  
}
```

Example (Another):

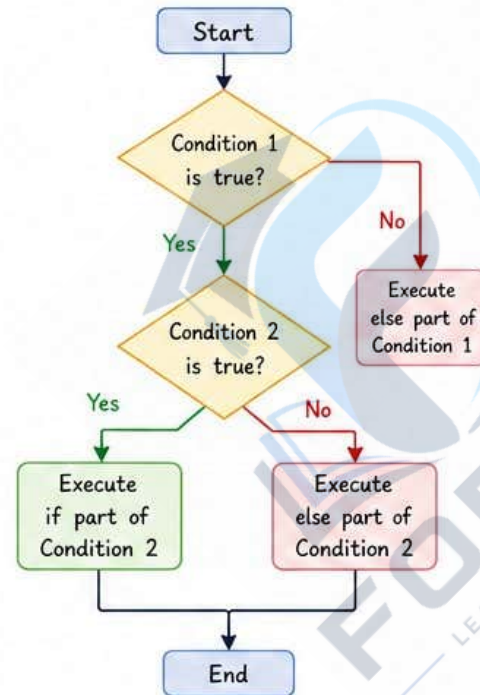
```
int age = 20;  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
} else {  
    System.out.println("Not eligible to vote");  
}
```

Output: Eligible to vote

2. Nesting of if...else statement

An **if...else** statement can be placed inside another **if...else** statement.

Flowchart:



Syntax:

```
if (condition1) {  
    if (condition2) {  
        // statements  
    }  
    else {  
        // statements  
    }  
}  
else {  
    // statements  
}
```

Example:

```
public class NestedIfExample {  
    public static void main(String[] args) {  
        int marks = 75;  
        if (marks >= 50) {  
            if (marks >= 75) {  
                System.out.println("Grade: Distinction");  
            } else {  
                System.out.println("Grade: First Class");  
            }  
        } else {  
            System.out.println("Grade: Fail");  
        }  
    }  
}
```

How it works:

- First, **condition1** is checked.
- If **condition1** is **true**, then **condition2** is checked.
 - If **condition2** is **true** → if part of **condition2** runs.
 - If **condition2** is **false** → **else** part of **condition2** runs.
- If **condition1** is **false** → **else** part of **condition1** runs.

Output for above example:

Grade: Distinction

★ Note:

Nesting can be multiple levels deep for complex decisions.



Decision Making and Branching in Java

What is else...if ladder?

It is used to check multiple conditions one by one.

As soon as one condition is **true**, its block is executed and the rest are **skipped**.

Else...if ladder

Used when we have multiple conditions to check.

1. Syntax:

```
if (condition1) {  
    // statements  
}  
else if (condition2) {  
    // statements  
}  
else if (condition3) {  
    // statements  
}  
.  
.  
.  
else {  
    // statements  
}
```

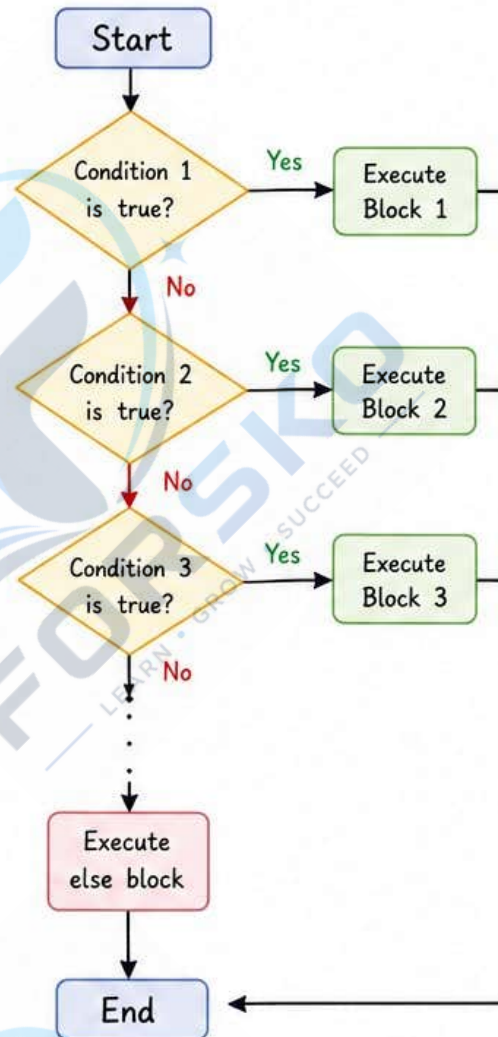
You can have any number of else if blocks.

else part is optional.

Example Use:

Used in grade calculation, fee slabs, age categories, discounts, etc.

2. Flowchart:



3. Example Program:

```
import java.util.Scanner;  
  
public class ElseIfLadderExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int marks;  
        System.out.print("Enter marks (0-100): ");  
        marks = sc.nextInt();  
  
        if (marks >= 90) {  
            System.out.println("Grade: A+");  
        }  
        else if (marks >= 75) {  
            System.out.println("Grade: A");  
        }  
        else if (marks >= 60) {  
            System.out.println("Grade: B");  
        }  
        else if (marks >= 40) {  
            System.out.println("Grade: C");  
        }  
        else {  
            System.out.println("Grade: Fail");  
        }  
        sc.close();  
    }  
}
```

4. Sample Output:

Enter marks (0-100): 92
Grade: A+

Enter marks (0-100): 68
Grade: B

Enter marks (0-100): 45
Grade: C

Enter marks (0-100): 35
Grade: Fail

5. Key Points:

- ★ Conditions are checked from top to bottom.
- ★ The first true condition's block is executed.
- ★ Once a block is executed, the rest are skipped.
- ★ else block runs only when no condition is true.

Decision Making and Branching in Java

Switch Statement

The **switch** statement is used to execute one block of code from multiple options based on the value of a variable/ expression.

1. Introduction

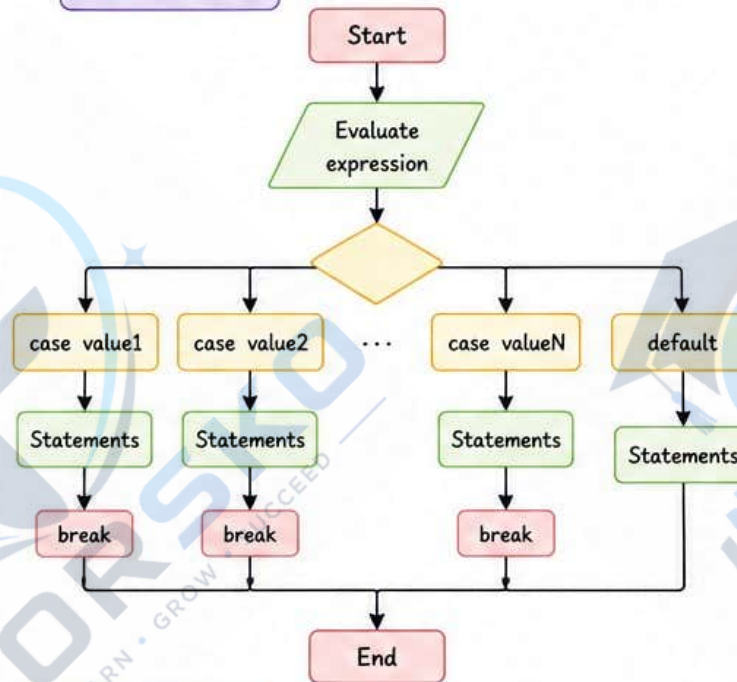
- Used when we have **multiple fixed values** to compare.
- It is more **readable** than multiple **else if** statements.
- Works with **byte, short, int, char, String, enum**.

2. Syntax

```
switch (expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    .  
    .  
    .  
    default:  
        // statements  
}
```

Optional
(default executes
if no case
matches)

3. Flowchart



4. Key Points

- ★ The expression is evaluated once.
- ★ The matching case block is executed.
- ★ **break** prevents the execution from "falling through" to next cases.
- ★ If **break** is omitted, the next case(s) will also execute.
- ★ Default block is optional.

5. Example Program

```
import java.util.Scanner;  
public class SwitchExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter day number (1-7): ");  
        int day = sc.nextInt();  
        String dayName;  
        switch (day) {  
            case 1: dayName = "Monday"; break;  
            case 2: dayName = "Tuesday"; break;  
            case 3: dayName = "Wednesday"; break;  
            case 4: dayName = "Thursday"; break;  
            case 5: dayName = "Friday"; break;  
            case 6: dayName = "Saturday"; break;  
            case 7: dayName = "Sunday"; break;  
            default: dayName = "Invalid day";  
        }  
        System.out.println("Day: " + dayName);  
        sc.close();  
    }  
}
```

6. Example Use

Used in menu-driven programs, **calculator**, month name finder, day name finder, grade calculator, etc.

7. Sample Output

Enter day number (1-7): **1**
Day: **Monday**

Enter day number (1-7): **5**
Day: **Friday**

Enter day number (1-7): **7**
Day: **Sunday**

Enter day number (1-7): **10**
Day: **Invalid day**

Decision Making and Looping: While statement

1. Introduction

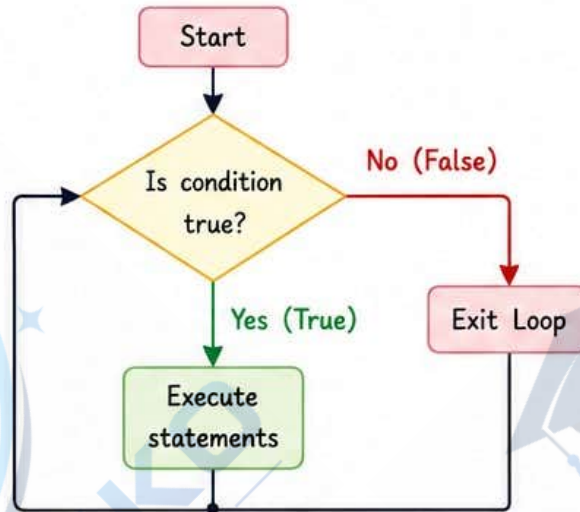
The **while** loop is an entry-controlled loop. It executes a block of code repeatedly as long as the given condition is **true**. When the condition becomes **false**, the loop terminates.

2. Syntax

```
while (condition) {  
    // statements  
}
```

Body of while loop executes repeatedly while condition is true.

3. Flowchart



4. Example Program

```
public class WhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        System.out.println("Numbers from 1 to 5:");  
        while (i <= 5) {           // condition  
            System.out.println(i); // loop body  
            i = i + 1;              // update  
        }  
        System.out.println("Loop finished.");  
    }  
}
```

5. How it works? (Step by Step)

1. Initialize $i = 1$
2. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 1, then i becomes 2.
3. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 2, then i becomes 3.
4. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 3, then i becomes 4.
5. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 4, then i becomes 5.
6. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 5, then i becomes 6.
7. Check condition $i \leq 5 \rightarrow \text{False}$
→ Exit loop.

6. Sample Output

Numbers from 1 to 5:

1
2
3
4
5

Loop finished.

Note:

The loop runs as long as the condition is true.

7. Example Use

- Reading input until user enters a valid value
- Processing items in a list until a condition is met
- Repeating a task until a specific goal is achieved

8. Key Points

- ★ The **while** loop checks the condition before executing the body.
- ★ If the condition is **false** at the beginning, the loop body will not execute at all.
- ★ Don't forget to update the variable inside the loop; otherwise, it may lead to an **infinite loop**.



Decision Making and Looping in Java

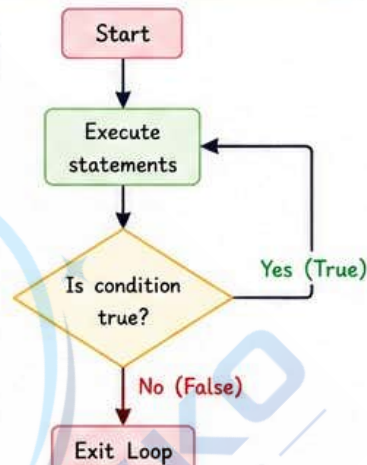
1. Do-While statement

The **do-while** loop is an **exit-controlled** loop. It executes the block of code **at least once**, and then repeats as long as the condition is **true**.

2. Syntax

```
do {  
    // statements  
} while (condition);
```

3. Flowchart



4. Example Program

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        System.out.println("Numbers from 1 to 5:");  
        do {  
            System.out.println(i);  
            i = i + 1;  
        } while (i <= 5);  
        System.out.println("Loop finished.");  
    }  
}
```

6. Key Points

- ★ The statements inside **do** block execute first, then the condition is checked.
- ★ The loop body executes **at least once** even if the condition is false initially.
- ★ Use **do-while** when you want the code to run once before checking the condition.

5. Sample Output

```
Numbers from 1 to 5:  
1  
2  
3  
4  
5  
Loop finished.
```

7. Example Use

- Menu driven programs
- User input validation
- Repeating a task until valid data is entered

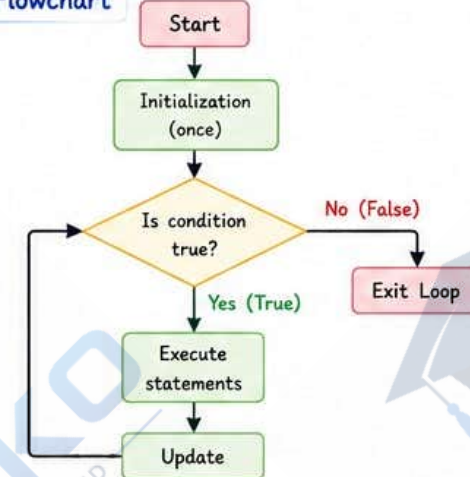
1. For statement

The **for** loop is an **entry-controlled** loop. It is used when the number of iterations (repetitions) is **known in advance**.

2. Syntax

```
for (initialization; condition; update) {  
    // statements  
}
```

3. Flowchart



4. Example Program

```
public class ForExample {  
    public static void main(String[] args) {  
        System.out.println("Numbers from 1 to 5:");  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
        System.out.println("Loop finished.");  
    }  
}
```

5. Sample Output

```
Numbers from 1 to 5:  
1  
2  
3  
4  
5  
Loop finished.
```

6. Key Points

- ★ Initialization runs once at the beginning.
- ★ Condition is checked **before** each iteration.
- ★ If condition is false initially, the loop body will not execute at all.
- ★ Update is executed after each iteration.

7. Example Use

- When number of iterations is known.
- Traversing arrays, lists, strings.
- Tables (e.g., multiplication table).
- Counting, summing, and processing a range of values.

8. Do-While vs For

Feature	Do-While	For
Type	Exit-controlled loop	Entry-controlled loop
When condition is checked	After executing the body	Before executing the body
Minimum executions	At least once	May execute zero times
Best used when	Iterations are not known, but at least one execution is required	Number of iterations is known in advance

Remember:

Choose the loop that best fits the logic and requirement of your program.



Decision Making and Looping in Java

Jump in Loop

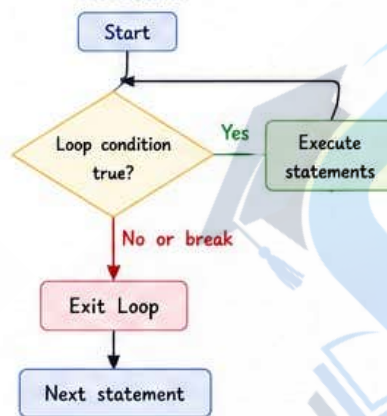
1. break Statement

The **break** statement is used to come out of the loop immediately, even if the loop condition is still true. After **break**, the program continues with the statement immediately after the loop.

Syntax:

```
break;
```

Flowchart:



Example Program:

```
public class BreakExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 6) { // exit when i becomes 6
                break;
            }
            System.out.print(i + " ");
        }
        System.out.println("\nLoop terminated.");
    }
}
```

Output:

```
1 2 3 4 5
Loop terminated.
```

Key Points:

- ★ **break** terminates the loop immediately.
- ★ Control comes out of the loop.

In loops, sometimes we need to skip the current iteration or terminate the loop immediately, Java provides two keywords for this purpose: **break** and **continue**.

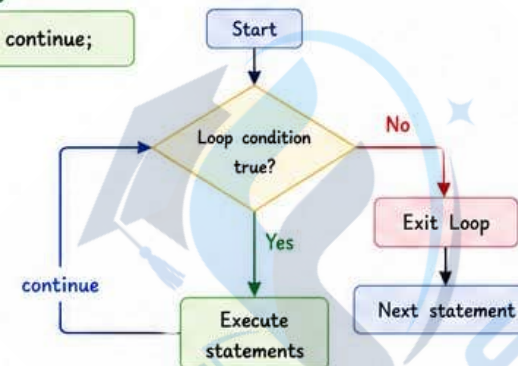
2. continue Statement

The **continue** statement is used to skip the current iteration and proceed to the next iteration of the loop. The remaining statements in the current iteration are not executed.

Syntax:

```
continue;
```

Flowchart:



Example Program:

```
public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) { // skip even numbers
                continue;
            }
            System.out.print(i + " ");
        }
        System.out.println("\nLoop finished.");
    }
}
```

Output:

```
1 3 5 7 9
Loop finished.
```

Key Points:

- ★ **continue** skips the current iteration.
- ★ Control remains in the loop for the next iteration.

4. Comparison

Feature	break	continue
Purpose	Terminates the loop immediately	Skips current iteration and continues
Where control goes	Comes out of the loop	Remains in the loop
Remaining statements in current iteration	Not executed	Not executed
Use when	You want to stop the loop completely	You want to skip some iterations

5. Notes

- ✓ **break** and **continue** can be used in for, while, and do-while loops.
- ✓ They affect only the innermost loop in case of nested loops.
- ✓ Use them carefully to make the program logic clear and efficient.



6. Example: Using both in same loop

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue; // skip even numbers
    if (i > 7) break; // exit when i > 7
    System.out.print(i + " ");
}
// Output: 1 3 5 7
```

How it works:

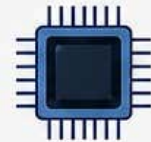
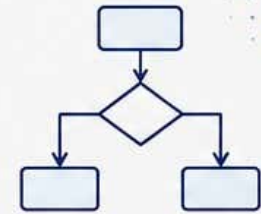
- i = 1 → print
- i = 2 → even → continue
- i = 3 → print
- i = 4 → even → continue
- i = 5 → print
- i = 6 → even → continue
- i = 7 → print
- i = 8 → > 7 → break → exit loop



Remember:

break is like saying "Stop the loop!"
continue is like saying "Skip this turn, I'll try next time!"

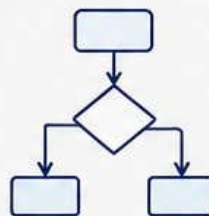
1010101010
0101010101
1010101010
0101010101



Unit - III



10110
01001
10101



CLASSES, OBJECTS AND METHODS

Java is an Object-Oriented Programming (OOP) language. The basic building blocks are **Classes**, **Objects** and **Methods**.

1. CLASS

- A class is a **blueprint** or **template** for creating objects.
- It defines **properties** (fields) and **behaviors** (methods) that objects of the class will have.

SYNTAX

```
class ClassName {  
    dataType field1;  
    dataType field2;  
    returnType method1() {  
        // code  
    }  
    returnType method2() {  
        // code  
    }  
}
```

Example

```
class Student {  
    String name;    // field  
    int age;        // field  
    void display() { // method  
        System.out.println(name + "-" + age);  
    }  
}
```

2. OBJECT

- An object is an **instance** of a class.
- It has its own identity, **state** (values of fields) and **behavior** (methods).
- Objects are created using **'new'** keyword.

CLASS

```
Student  
- name  
- age  
+ display()
```

OBJECTS (Instances)

s1

```
name = "Ali"  
age = 20  
display()
```

s2

```
name = "Sara"  
age = 22  
display()
```

SYNTAX

```
ClassName objRef = new ClassName();
```

Example

```
Student s1 = new Student();  
s1.name = "Ali";  
s1.age = 20;  
s1.display();
```

3. METHOD

- A **method** is a block of code that **performs a specific task**.
- Methods define the **behavior** of the objects.
- Methods can take **parameters** and can **return a value**.

SYNTAX

```
returnType methodName(parameters) {  
    // code  
    return value; // optional  
}
```

TYPES OF METHODS

- With return value
- Without return value (void)
- With parameters
- Without parameters

★ REAL LIFE EXAMPLE

Class : Car (blueprint)
Objects : car1, car2 (instances)
Methods : start(), stop(), move()
(behaviors)



RELATIONSHIP BETWEEN THEM



CLASSES, OBJECTS AND METHODS: INTRODUCTION

Java is an **Object-Oriented Programming (OOP)** language.
The basic building blocks of OOP are **Classes**, **Objects** and **Methods**.

1. CLASS



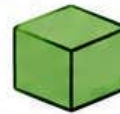
- A class is a **blueprint** or **template** for creating objects.
- It defines the **properties** (fields) and **behaviors** (methods) that the objects of the class will have.

Analogy

A class is like a design of a house. From the same design, we can build many houses (objects).



2. OBJECT



- An object is an **instance** of a class.
- It has its own **identity**, **state** (values of fields) and **behavior** (methods).
- Objects are created using the **'new'** keyword.

Analogy

Objects are like real houses built from the same design.



3. METHOD



- A method is a block of code that performs a **specific task**.
- Methods define the **behavior** of the objects.
- Methods can take **parameters** and can **return a value**.

Analogy

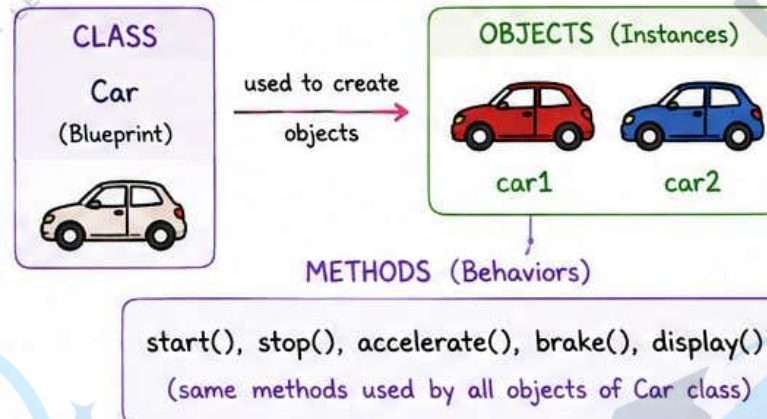
Methods are like the functions of a device that tell it what to do.



KEY POINTS

- ★ Class → Blueprint
- ★ Object → Real world entity (an instance)
- ★ Method → Behavior (action performed by object)
- ★ Many objects can be created from one class.
- ★ Each object has its own data in fields, but shares the methods of the class.

REAL LIFE EXAMPLE



WHY USE CLASSES, OBJECTS AND METHODS?

- Code Reusability
- Better Organization
- Data Hiding and Security
- Easy Maintenance
- Real world Modeling

In short: **Classes** define what an object is, **Objects** represent real entities, and **Methods** define what an object can do.

DEFINING A CLASS, DECLARATION OF FIELDS, METHODS

In Java, a **class** is a user-defined **blueprint** from which objects are created.

1. DEFINING A CLASS

- A class is defined using the keyword **class**.
- It contains fields (data) and methods (behavior).

SYNTAX

```
class ClassName {  
    // fields (data members)  
    // methods (member functions)  
}
```

EXAMPLE

```
class Student {  
}
```

→ This is a simple class named Student



A class is just a template or blueprint. No memory is allocated when a class is defined.

COMPLETE EXAMPLE

```
class Student {  
    // Fields  
    int rollNo;  
    String name;  
    double marks;  
    // Method  
    void display() {  
        System.out.println(name + " " + rollNo + " " + marks);  
    }  
}
```

Declaration of Fields

Method

2. DECLARATION OF FIELDS

- Fields (or data members) are variables declared inside a class.
- They represent the state or properties of an object.

SYNTAX

```
dataType fieldName; // declaration  
accessModifier dataType fieldName;
```

EXAMPLE

```
class Student {  
    int rollNo;    // field 1  
    String name;   // field 2  
    double marks;  // field 3  
}
```

These are fields of class Student



Fields hold data. Each object will have its own copy of these fields.

3. METHODS

- Methods are blocks of code that perform specific tasks.
- They define the behavior of the objects.

SYNTAX

```
returnType methodName(parameters) {  
    // statements  
}
```

EXAMPLE

```
class Student {  
    int rollNo;  
    String name;  
    double marks;  
    void display() {  
        System.out.println(name + " " + rollNo + " " + marks);  
    }  
}
```

→ display() is a method of class Student



Methods perform actions. They can use fields and other methods inside the class.

KEY POINTS

- ★ A class is defined using the keyword **class**.
- ★ Fields store the data (state) of objects.
- ★ Methods define the behavior (actions) of objects.
- ★ Fields and methods are declared inside the class.
- ★ Objects are created from a class and can access its fields and methods.

CLASS → OBJECT RELATIONSHIP

CLASS
(Blueprint)
Student

creates →

OBJECTS
(Instances)
s1, s2, s3, ...

Each object has its own values in fields but shares the methods of the class.

OBJECTS CREATION AND ACCESSING CLASS MEMBERS

In Java, **objects** are created from classes. After creating objects, we can **access** (use) their fields and methods using the **dot (.)** operator.

1. OBJECTS CREATION

- An object is an **instance** of a class.
- Objects are created using the **'new'** keyword.
- Each object has its own copy of fields but shares the methods of the class.

SYNTAX

```
ClassName objectRef = new ClassName();
```

Reference
(variable)

Object created

Creating Objects

```
Student s1 = new Student(); // object 1
Student s2 = new Student(); // object 2
```

Memory View (Conceptual)

Class Student
(Blueprint)

Fields:

int rollNo
String name
double marks

Method:

display()

creates

Object s1

rollNo → 0
name → null
marks → 0.0

Object s2

rollNo → 0
name → null
marks → 0.0

Each object has its own copy of fields.
Methods are shared by all objects.

CLASS
(Blueprint)

```
Student
int rollNo;
String name;
double marks;
void display();
```

OBJECTS (Instances)



s1



s2

EXAMPLE

```
class Student {
    int rollNo;
    String name;
    double marks;
    void display() {
        System.out.println(name + " " +
            rollNo + " " + marks);
    }
}
```

2. ACCESSING CLASS MEMBERS

- Fields and methods of an object are accessed using the **dot (.)** operator.
- We can **read/change** the values of fields.
- We can **call methods** to perform actions.

ACCESSING FIELDS

objectRef.fieldName // to read or write

Example:

```
s1.rollNo = 101; // write value
s1.name = "Amit"; // write value
s1.marks = 88.5; // write value

int r = s1.rollNo; // read value
String n = s1.name; // read value
```

ACCESSING METHODS

objectRef.methodName(arguments);

Example:

```
s1.display(); // call method
s2.display();
```



Fields represent data (state).
Methods represent behavior (actions).

COMPLETE EXAMPLE

```
class Student {
    int rollNo;
    String name;
    double marks;
    void display() {
        System.out.println(name + " " + rollNo +
            " " + marks);
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Student s1 = new Student(); // create object
        s1.rollNo = 101; // access field
        s1.name = "Amit"; // access field
        s1.marks = 88.5; // access field
        s1.display(); // call method
    }
}
```

REAL LIFE ANALOGY



Class
(Blueprint)



Object 1
(House 1)



Object 2
(House 2)

The blueprint (class) defines the design.
Many houses (objects) can be built from it.
Each house has its own features (data)
but follows the same design (methods).

METHOD OVERLOADING

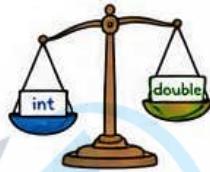
Method Overloading is a feature of Java that allows a class to have more than one method with the **same name** but **different parameter lists**.

KEY POINTS

- ★ Same method name
- ★ Different number of parameters
- ★ or Different type of parameters
- ★ or Different order of parameters
- ★ Return type may be same or different.
- ★ It improves code readability.

1. WHY USE METHOD OVERLOADING?

- Increases readability of the program.
- Provides flexibility to the programmer.
- Allows a method to perform similar operations on different kinds of data or in different ways.



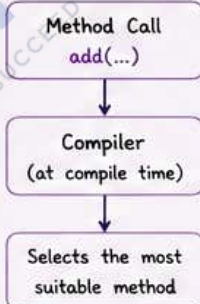
2. SYNTAX

```
returnType methodName(parameter list 1) {  
    // body  
}  
returnType methodName(parameter list 2) {  
    // body  
}  
returnType methodName(parameter list 3) {  
    // body  
}  
.....
```

3. EXAMPLE

```
class Calculator {  
    // method 1: no parameter  
    int add() {  
        return 0;  
    }  
  
    // method 2: two int parameters  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // method 3: three int parameters  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // method 4: two double parameters  
    double add(double x, double y) {  
        return x + y;  
    }  
}
```

4. HOW OVERLOADING WORKS?



- When a method is called, the compiler checks the arguments.
- It matches the method whose parameter list matches the arguments.
- The selected method is executed.

5. EXAMPLE WITH OUTPUT

```
class Test {  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
  
        System.out.println(c.add());           // 0  
        System.out.println(c.add(10, 20));     // 30  
        System.out.println(c.add(10, 20, 30)); // 60  
        System.out.println(c.add(5.5, 4.5));   // 10.0  
    }  
}
```

Output:

```
0  
30  
60  
10.0
```

6. RULES FOR METHOD OVERLOADING

- Methods must have the same name.
- Parameter list must be different by:
 - Number of parameters
 - or Type of parameters
 - or Order of parameters
- Return type may be same or different.
- Access specifier may be same or different.
- It is determined at compile time (static binding).



7. IMPORTANT NOTE



- Overloading is not possible by changing only the return type.
`int area(int r) { ... }`
`double area(int r) { ... }` × Not allowed (same parameters)

REAL LIFE ANALOGY



A TV remote has the same button name (e.g., Volume) but works differently based on how we use it:

- `Volume()` → increase by 1
- `Volume(int level)` → set to a level
- `Volume(int level, boolean mute)` → set level and mute/unmute

Same name, different ways of use!

CONSTRUCTOR AND ITS TYPES

Constructor is a special method that is used to **initialize objects** of a class. It has the **same name** as the class, has **no return type** (not even **void**) and is called automatically when an object is created using the **new** keyword.

KEY POINTS

- ★ Constructor is used to initialize objects.
- ★ It is called automatically at the time of object creation.
- ★ Constructor name = class name.
- ★ It has no return type.
- ★ Constructors can be overloaded.

1. WHY CONSTRUCTOR?

- Initializes objects when they are created.
- Helps to set initial values to data members.
- Improves code reusability.
- It is called only once per object.



If we do not write any constructor, Java provides a **default constructor** automatically (no-arg constructor).

① DEFAULT CONSTRUCTOR (No-arg Constructor)

- It has no parameters.
- If we do not write any constructor, Java provides it automatically.

Example:

```
class Student {
    int rollNo;
    String name;
    Student() { // default constructor
        rollNo = 0;
        name = "Unknown";
    }
}
```

Usage:

```
Student s = new Student();
System.out.println(s.rollNo);
System.out.println(s.name);
```

Output:

0
Unknown

2. SYNTAX

```
class ClassName {
    ClassName() {
        // constructor body
    }
}
```

Constructor name
(same as class name)

No return type
(not even void)

3. TYPES OF CONSTRUCTORS

There are three types of constructors in Java.

- ① Default Constructor (No-arg Constructor)
- ② Parameterized Constructor
- ③ Copy Constructor (User-defined)

② PARAMETERIZED CONSTRUCTOR

- It has parameters.
- Used to initialize objects with different values.

Example:

```
class Student {
    int rollNo;
    String name;
    Student(int r, String n) { // parameterized
        rollNo = r;
        name = n;
    }
}
```

Usage:

```
Student s = new Student(101, "Amit");
System.out.println(s.rollNo + " " + s.name);
```

Output:

101 Amit

③ COPY CONSTRUCTOR (User-defined)

- It takes an object of the same class as a parameter.
- Used to copy the values of one object into another.

Example:

```
class Student {
    int rollNo;
    String name;
    Student(Student s) { // copy constructor
        rollNo = s.rollNo;
        name = s.name;
    }
}
```

Usage:

```
Student s1 = new Student(101, "Amit");
Student s2 = new Student(s1);
System.out.println(s2.rollNo + " " + s2.name);
```

Output:

101 Amit

(Here, Student(int, String) is a parameterized constructor used to create s1.)

REAL LIFE ANALOGY



Default Constructor
Like a house with default furniture.



Parameterized Constructor
Like a house where we choose furniture.



Copy Constructor
Like copying furniture from one house to another.

COMPARISON OF CONSTRUCTOR TYPES

Type	Parameters	Provided By	Purpose
Default (No-arg)	No	JVM (if not written)	Initializes with default values
Parameterized	Yes	Programmer	Initializes with given values
Copy (User-defined)	Object of same class	Programmer	Initializes by copying values from another object

IMPORTANT NOTES

- Constructor cannot be abstract, final, static or synchronized.
- Constructor is not inherited.
- Multiple constructors can exist in a class (overloading).
- If we write any constructor, default constructor is not provided automatically.

CONSTRUCTOR OVERLOADING

Constructor Overloading is a feature in Java that allows a class to have more than one constructor with **same name** but **different parameter list**.

KEY POINTS

- ★ Multiple constructors with same name.
- ★ Difference in number, type or order of parameters.
- ★ Return type is not considered.
- ★ It helps to initialize objects in different ways.
- ★ Improves code flexibility and readability.

1. WHY USE CONSTRUCTOR OVERLOADING?

- Allows initialization of objects in multiple ways.
- Provides flexibility to the programmer.
- Improves code readability.
- Avoids the need for multiple class names.



2. SYNTAX

```
class ClassName {  
    ClassName() {  
        // constructor body  
    }  
    ClassName(parameter list 1) {  
        // constructor body  
    }  
    ClassName(parameter list 2) {  
        // constructor body  
    }  
}
```

Default Constructor

Overloaded Constructors

3. EXAMPLE

```
class Student {  
    int rollNo;  
    String name;  
    double marks;  
  
    // Constructor 1: no-arg constructor  
    Student() {  
        rollNo = 0;  
        name = "Unknown";  
        marks = 0.0;  
    }  
  
    // Constructor 2: one parameter  
    Student(int r) {  
        rollNo = r;  
        name = "Unknown";  
        marks = 0.0;  
    }  
  
    // Constructor 3: two parameters  
    Student(int r, String n) {  
        rollNo = r;  
        name = n;  
        marks = 0.0;  
    }  
  
    // Constructor 4: three parameters  
    Student(int r, String n, double m) {  
        rollNo = r;  
        name = n;  
        marks = m;  
    }  
}
```

4. EXAMPLE WITH OBJECT CREATION

```
public class Test {  
    public static void main(String[] args) {  
        // Using no-arg constructor  
        Student s1 = new Student();  
        // Using one parameter constructor  
        Student s2 = new Student(101);  
        // Using two parameter constructor  
        Student s3 = new Student(102, "Amit");  
        // Using three parameter constructor  
        Student s4 = new Student(103, "Neha", 87.5);  
    }  
}
```

5. OUTPUT

Object	Constructor Used	rollNo	name	marks
s1	Student()	0	Unknown	0.0
s2	Student(int)	101	Unknown	0.0
s3	Student(int, String)	102	Amit	0.0
s4	Student(int, String, double)	103	Neha	87.5

6. HOW IT WORKS?

When an object is created, the compiler looks at the arguments passed.

It matches the constructor whose parameter list matches the arguments.

The matched constructor is executed to initialize the object.

Same name
Different
parameter list



7. RULES FOR CONSTRUCTOR OVERLOADING

- ✓ Constructor name must be same as the class name.
- ✓ Parameter list must be different by:
 - Number of parameters
 - Type of parameters
 - Order of parameters
- ✓ Return type may be same or different (but return type cannot be used to overload).
- ✓ Access specifier may be same or different.

REAL LIFE ANALOGY

A restaurant has the same dish (e.g., Pizza) but offers it in different sizes and with different toppings.



Small Pizza

Pizza with extra cheese

Pizza with cheese and veggies

Pizza with cheese, veggies and olives

Same name (Pizza)
Different options
(parameter list)

8. IMPORTANT NOTE

- ★ Constructors cannot be abstract, final, static or synchronized.
- ★ They are not inherited.
- ★ If no constructor is written, Java provides a default no-arg constructor automatically.
- ★ Constructor Overloading is achieved at compile time.

USE OF new, delete & this KEYWORD

These keywords are used with classes and objects to manage memory and access members.

1. USE OF new KEYWORD

The new keyword is used to allocate memory dynamically at run time and to create objects in heap memory.



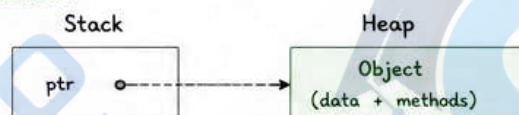
SYNTAX

```
ClassName *obj = new ClassName();
```

↓ ↓

pointer to object memory allocated & object created

DIAGRAM



EXAMPLE

```
class Student {
public:
    int rollNo;
    string name;
    Student() {
        rollNo = 0;
        name = "Unknown";
    }
};
```

// object created using new

```
Student *s = new Student();
s->rollNo = 101;
s->name = "Amit";
cout << s->rollNo << " "
      << s->name;
```

- ✓ new returns the address of the newly created object. Memory remains allocated until we use delete.

2. USE OF delete KEYWORD

The delete keyword is used to free the memory allocated by new. It destroys the object and deallocates memory.

SYNTAX

```
delete pointer_variable;
```

↓

frees memory and destroys the object



EXAMPLE

```
Student *s = new Student(); // memory allocated
s->rollNo = 101;
s->name = "Amit";

delete s; // memory freed
s = NULL; // good practice
```

NOTE

- After using delete, the object is destroyed.
- The memory can be reused by the program.
- Set the pointer to NULL to avoid dangling pointer.

LIFE CYCLE



- ⚠ If we do not use delete, memory will not be freed and it may lead to memory leak.

3. USE OF this KEYWORD

The this keyword is a pointer that refers to the current object. It is used to access current class members.

SYNTAX

```
this->member; // for data members
this->method(); // for member functions
```

this refers to the current object.

EXAMPLE 1: Resolving name conflict

```
class Student {
    int rollNo;
    string name;
public:
    Student(int rollNo, string name) {
        this->rollNo = rollNo; // current object's rollNo
        this->name = name; // current object's name
    }
};
```

Parameter names are same as data members. this is used to differentiate.

EXAMPLE 2: Using this with member functions

```
class Counter {
    int count;
public:
    Counter() { count = 0; }
    void increment() { this->count++; }
    void display() { cout << "Count = " << this->count; }
};
```

this is used to access current object's data member.

KEY POINTS SUMMARY

new

- ✓ Allocates memory at run time in heap.
- ✓ Creates an object and returns its address.
- ✓ Syntax: ptr = new ClassName();
- ✓ Must be released using delete.



delete

- ✓ Deallocates memory allocated by new.
- ✓ Destroys the object.
- ✓ Syntax: delete ptr;
- ✓ Prevents memory leak.



this

- ✓ Refers to the current object.
- ✓ Used to access current object's members.
- ✓ Resolves name conflicts.
- ✓ Syntax: this->member;



REAL LIFE ANALOGY



- new → Building a new house (allocating memory)
- delete → Demolishing the house (freeing memory)
- this → Referring to "this house" (current object)

ACCESS SPECIFIER: public, private & protected

Access specifiers (or access modifiers) control the **visibility** (scope) of class members (fields, methods, constructors and inner classes). They decide which parts of a program can access a member.

1. public

- Members declared as **public** can be accessed from anywhere in the program.
- There is no restriction on visibility.

Example:

```
public class Demo {  
    public int data = 10;  
    public void show() {  
        System.out.println("This is public");  
    }  
}
```

Accessible from:

- ✓ Same class
- ✓ Same package
- ✓ Subclasses (even in different package)
- ✓ Other classes (anywhere)

2. private

- Members declared as **private** can be accessed only within the **same class**.
- Not accessible outside the class.

Example:

```
public class Demo {  
    private int data = 20; // only in this class  
    private void show() {  
        System.out.println("This is private");  
    }  
}
```

Accessible from:

- ✓ Same class only

3. protected

- Members declared as **protected** can be accessed:
 - Within the same package.
 - In subclasses (even if they are in a different package).

Example:

```
public class Demo {  
    protected int data = 30;  
    protected void show() {  
        System.out.println("This is protected");  
    }  
}
```

Accessible from:

- ✓ Same class
- ✓ Same package
- ✓ Subclasses (even in different package)
- ✗ Other classes in different package (Not allowed)

4. COMPARISON TABLE

Access Specifier	Same Class	Same Package	Subclasses in Different Package	Other Classes in Different Package
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
private	✓	✗	✗	✗

✓ = Accessible ✗ = Not Accessible



Key Points

- If no access specifier is written, the default access is package-private (accessible within the same package only).
- We cannot use private or protected at the class level.
- Top-level classes can be public or default (package-private) only.

5. EXAMPLE PROGRAM

Class: MyClass (in package pack1)

```
package pack1;  
  
public class MyClass {  
    public int a = 1;  
    protected int b = 2;  
    private int c = 3;  
    int d = 4; // default (package-private)  
  
    public void show() {  
        System.out.println("a=" + a);  
        System.out.println("b=" + b);  
        System.out.println("c=" + c);  
        System.out.println("d=" + d);  
    }  
}
```

Class in same package (pack1)

```
package pack1;  
  
public class SamePackage {  
    public void test() {  
        MyClass obj = new MyClass();  
        System.out.println(obj.a); // ok  
        System.out.println(obj.b); // ok  
        // System.out.println(obj.c); // error  
        System.out.println(obj.d); // ok  
    }  
}
```

Subclass in different package (pack2)

```
package pack2;  
import pack1.MyClass;  
  
public class SubClass extends MyClass {  
    public void test() {  
        MyClass obj = new MyClass();  
        System.out.println(obj.a); // ok (public)  
        System.out.println(obj.b); // ok (protected)  
        // System.out.println(obj.c); // error (private)  
        // System.out.println(obj.d); // error (default)  
    }  
}
```

- ★ **private** is used for data hiding (encapsulation) to protect data from outside access.
- protected** is used for inheritance purpose.
- public** is used when the member needs to be accessible everywhere.

6. REAL LIFE ANALOGY

public



Like a public park.
Anyone can enter from anywhere.

protected



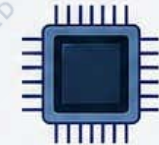
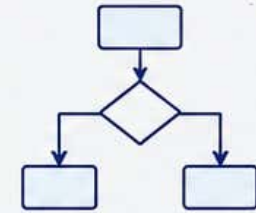
Like family area.
Family members and relatives (subclasses) can access.

private



Like a locked room.
Only the owner (class itself) can access.

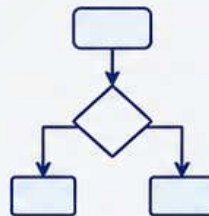
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101



INHERITANCE: INTRODUCTION

Inheritance is one of the fundamental concepts of Object-Oriented Programming (OOP).

It is a mechanism in which a new class (derived or child class) acquires the properties (fields) and behaviors (methods) of an existing class (base or parent class).

KEY POINTS

- ★ Promotes code reusability.
- ★ Establishes a relationship between classes.
- ★ Provides a way to extend existing classes.
- ★ Improves maintainability and reduces development time.
- ★ Supports method overriding (run-time polymorphism).

1. WHAT IS INHERITANCE?

- Inheritance allows a class to inherit fields and methods from another class.
- The class whose members are inherited is called the base (super) class.
- The class that inherits is called the derived (sub) class.
- It represents an "is-a" relationship.

Example:

A Dog is an Animal.

↓
Dog (derived class) inherits properties of Animal (base class).



2. SYNTAX

```
class BaseClass {  
    // fields and methods  
};  
  
class DerivedClass extends BaseClass {  
    // additional fields and methods  
};
```

Here,

- DerivedClass (child) extends BaseClass (parent).
- The child class can use members of the parent class.
- Use of **extends** keyword indicates inheritance.

3. HOW IT WORKS?

Parent class is defined.

↓
Child class is created using **extends**.

↓
Child class inherits fields and methods of parent class.

↓
Child class can use them directly or extend them.

- The child class automatically gets accessible members of the parent class (except private ones).
- It can add new members and methods.
- It can also modify behavior of inherited methods (method overriding).
- This helps in code reusability and better organization.

4. EXAMPLE

```
// Parent (Base) class  
class Animal {  
    String name;  
    void eat() {  
        System.out.println(name + " eats food.");  
    }  
}  
  
// Child (Derived) class  
class Dog extends Animal {  
    void bark() {  
        System.out.println(name + " barks.");  
    }  
}
```

Usage:

```
public class Test {  
    public static void main(String[] args){  
        Dog d = new Dog();  
        d.name = "Buddy";  
        d.eat(); // inherited method  
        d.bark(); // own method  
    }  
}
```

Output:

Buddy eats food.
Buddy barks.

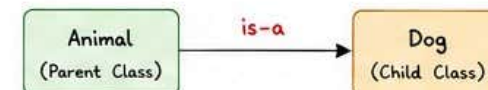
5. TYPES OF INHERITANCE

Depending on the number of classes involved:

- ① Single Inheritance - One child class inherits from one parent class.
- ② Multilevel Inheritance - A derived class inherits from another derived class.
- ③ Hierarchical Inheritance - Multiple derived classes inherit from one base class.
- ④ Multiple Inheritance* - A class inherits from more than one class. (*Not supported in Java with classes, but possible with interfaces.)
- ⑤ Hybrid Inheritance - Combination of two or more types.

6. BENEFITS

- ✓ Code reusability.
- ✓ Method overriding allows run-time polymorphism.
- ✓ Better organization and hierarchical classification.
- ✓ Easy maintenance and extension of programs.
- ✓ Supports the "is-a" relationship.



7. REAL LIFE ANALOGY



- Vehicle is the parent class.
- Car, Bike, Truck are child classes.
- All vehicles have some common properties (wheels, engine, etc.)
- Each type has its own specific features.
- This is inheritance in real life!



IMPORTANT NOTE

- Constructors, private members and final methods are not inherited.
- The child class can access public and protected members of the parent class.
- Inheritance promotes the principle: "Write once, use many times."

TYPES OF INHERITANCE

In OOP, inheritance allows a new class (child/derived class) to acquire the properties (data members) and behaviors (member functions) of an existing class (parent/base class). Depending on the arrangement of classes, inheritance can be of different types.

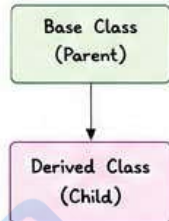
KEY POINTS

- ★ Inheritance represents an "is-a" relationship.
- ★ It promotes code reusability and extensibility.
- ★ The child class can use the members of parent class.
- ★ Java supports all types of inheritance (through classes and interfaces).

1 SINGLE INHERITANCE

- A child class inherits from a single parent class.
- It is the simplest form of inheritance.

Diagram:



Example:

```

class Animal {
    void eat() { ... }
}

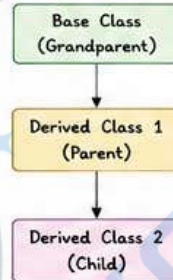
class Dog extends Animal {
    void bark() { ... }
}

Dog is-an Animal.
  
```

2 MULTILEVEL INHERITANCE

- A derived class inherits from another derived class.
- Forms a chain of inheritance.

Diagram:



Example:

```

class Animal {
    void eat() { ... }
}

class Mammal extends Animal {
    void walk() { ... }
}

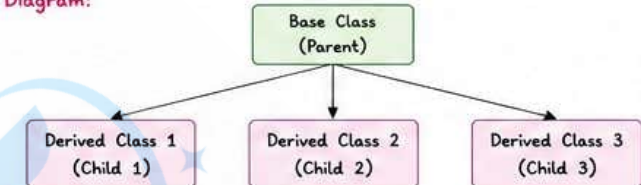
class Dog extends Mammal {
    void bark() { ... }
}

Dog is-a Mammal, Mammal is-an Animal.
  
```

3 HIERARCHICAL INHERITANCE

- More than one derived class inherits from a single parent class.
- All child classes are at the same level.

Diagram:



Example:

```

class Shape {
    void display() { ... }
}

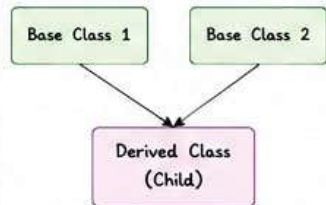
class Circle extends Shape { ... }
class Rectangle extends Shape { ... }
class Triangle extends Shape { ... }

Circle, Rectangle and Triangle are Shapes.
  
```

4 MULTIPLE INHERITANCE

- A class inherits from more than one parent class.
- (Note: Java does not support multiple inheritance with classes, but it can be achieved using interfaces.)

Diagram:



Example (Using Interfaces in Java):

```

interface I1 { void m1(); }
interface I2 { void m2(); }

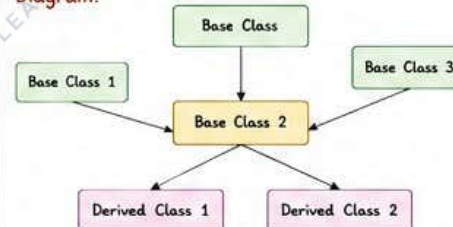
class Child implements I1, I2 {
    public void m1() { ... }
    public void m2() { ... }
}

Child gets features of I1 and I2.
  
```

5 HYBRID INHERITANCE

- Combination of two or more types of inheritance.
- Example: Combination of hierarchical and multiple inheritance.
- (Not supported directly with classes in Java, but possible using interfaces.)

Diagram:



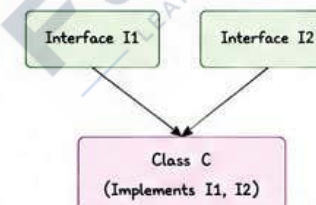
Explanation:

Base Class 1, Base Class 2 and Base Class 3 are combined in such a way that Derived Class 1 and Derived Class 2 get features from more than one source.

6 MULTIPLE INHERITANCE (THROUGH INTERFACES)

- A class implements more than one interface.
- This is a form of multiple inheritance in Java.

Diagram:



Example:

```

interface I1 { void m1(); }
interface I2 { void m2(); }

class C implements I1, I2 {
    public void m1() { ... }
    public void m2() { ... }
}

Class C gets features of both I1 and I2.
  
```

SUMMARY TABLE

Type	Parent Classes	Child Classes	Structure	Example
1. Single	1	1	Linear	Animal → Dog
2. Multilevel	1 per level	1	Chain	Animal → Mammal → Dog
3. Hierarchical	1	Many	Tree	Shape → Circle, Rectangle, Triangle
4. Multiple	Many	1	Converging	I1, I2 → Child (via interfaces)
5. Hybrid	Many	Many	Combination	Combination of above types

IMPORTANT NOTE



- Java does not support multiple inheritance with classes (to avoid ambiguity).
- It supports it through interfaces.
- Private members and constructors are not inherited.
- The child class can access public and protected members of the parent class.

1. SUPER & EXTENDED CLASS

In inheritance, the relationship between classes is created using the keyword **extends** and **super** and some values can be made constant using the keyword **final**.

1. SUPER & EXTENDED CLASS

A. EXTENDED CLASS (using extends)

- The keyword **extends** is used to extend (inherit) a class.
- The class which is inherited is called **Super class** (Parent class / Base class).
- The class which inherits is called **Sub class** (Child class / Derived class).

Syntax :

```
class SuperClass {  
    // fields and methods  
}  
class SubClass extends SuperClass {  
    // additional fields and methods  
}
```

Example :

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
// Usage  
Dog d = new Dog();  
d.eat();    // Inherited from Animal  
d.bark();   // Own method of Dog
```

B. SUPER KEYWORD (using super)

- **super** is a reference variable which refers to the immediate parent class object.
- It is used to access parent class members (variables, methods and constructors).

Uses of **super** :

1. Access parent class variable
2. Access parent class method
3. Call parent class constructor

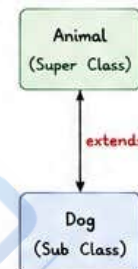
Example 1: super to access parent class method

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
class Dog extends Animal {  
    void eat() {  
        super.eat();    // calls parent class eat()  
        System.out.println("Dog also eats");  
    }  
}
```

Example 2: super to call parent class constructor

```
class Animal {  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
}  
class Dog extends Animal {  
    Dog() {  
        super();    // calls parent class constructor  
        System.out.println("Dog constructor");  
    }  
}
```

Relationship



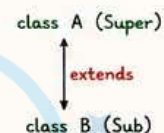
Note:

- **super** can only be used in a subclass.
- It always refers to the immediate parent class.

KEY POINTS SUMMARY

SUPER & EXTENDED

- ✓ **extends** is used to inherit (create child class).
- ✓ Parent class is also called Super class (Base class).
- ✓ Child class is also called Sub class (Derived class).
- ✓ **super** refers to immediate parent class object.
- ✓ **super** is used to access parent class members and constructors.



B extends A
B is a child of A
A is a parent of B

KEY POINTS

- ★ **extends** is used to inherit a class.
- ★ **super** is used to access parent class members.
- ★ **final** variable cannot be changed once initialized.
- ★ It helps in security, consistency and reliability.

2. FINAL VARIABLE

- **final** is a non-access modifier.
- A **final** variable (constant) cannot be changed once it is initialized.
- It is like a constant whose value remains fixed.

Syntax :

```
final data_type VARIABLE_NAME = value;
```

Examples :

1. final variable

```
class Test {  
    final int MAX = 100;  
    void show() {  
        System.out.println("MAX = " + MAX);  
    }  
}  
// MAX = 100;    // cannot be changed  
// MAX = 200;    // Error
```

2. final with reference (object)

```
class Test {  
    final String name = "Java";  
    final int[] arr = {1, 2, 3};  
    void show() {  
        // name = "Python";    // Error  
        arr[0] = 10;           // Allowed  
    }  
}  
We cannot change the reference of arr,  
but we can change its contents.
```

Why use final variable ?

- ✓ To make values constant (cannot be changed).
- ✓ To prevent accidental modification.
- ✓ To improve security and reliability.
- ✓ To represent fixed values (like PI, MAX_SIZE, etc.).

Example (Constant)

```
class MathDemo {  
    final double PI = 3.14159;  
    final int DAYS_IN_WEEK = 7;  
}
```

FINAL VARIABLE

- ✓ **final** makes a variable constant.
- ✓ Its value cannot be changed once initialized.
- ✓ Makes code safe, consistent and easy to maintain.
- ✓ Name of final variable is usually in UPPER_CASE.



Use final for values that must remain unchanged!

METHOD & CLASS

In OOP, a **class** is a blueprint or template that defines data (fields) and behavior (methods). A **method** is a block of code that performs a specific task when called.

KEY POINTS

- ★ A class can have fields (variables) and methods.
- ★ A method defines the behavior of an object.
- ★ Methods are called to perform operations.
- ★ A class is a user-defined data type.
- ★ Objects are created from classes.

1. METHOD

- A method is a collection of statements that performs a specific task.
- It is defined inside a class.
- It is called (invoked) using an object.
- It may or may not return a value.

Syntax :

```
[access_modifier] return_type methodName(parameters) {  
    // body of the method  
}
```

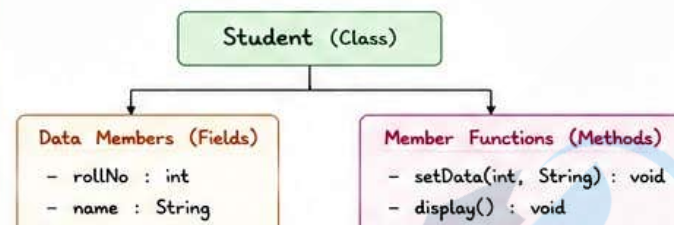
Example :

```
class Demo {  
    void greet() {           // method without return value  
        System.out.println("Hello! Welcome");  
    }  
    int add(int a, int b) {   // method with return value  
        int sum = a + b;  
        return sum;  
    }  
}
```

Types of Methods

- ① User-defined methods - Created by the programmer.
- ② Built-in methods - Provided by Java API (e.g., println(), length()).
- ③ Static methods - Belong to the class, called using class name.
- ④ Non-static (instance) methods - Belong to objects.

4. CLASS DIAGRAM (Conceptual View)



2. CLASS

- A class is a blueprint or template for creating objects.
- It defines fields (data) and methods (behavior).
- It does not occupy memory until objects are created.
- Objects are instances of a class.

Syntax :

```
class ClassName {  
    // fields (variables)  
    data_type variable1, variable2;  
  
    // methods  
    return_type methodName(parameters) {  
        // body  
    }  
}
```

Example :

```
class Student {           // class  
    // fields  
    int rollNo;  
    String name;  
  
    // method  
    void display() {  
        System.out.println("RollNo: " + rollNo);  
        System.out.println("Name: " + name);  
    }  
}
```

5. EXPLANATION

- A class groups data and methods together.
- Fields store the state (data).
- Methods define the behavior (operations).
- Objects are created using the class.
- Methods are called through objects using dot (.) operator.

3. USING METHOD IN A CLASS

```
class Student {  
    int rollNo;  
    String name;  
  
    void setData(int r, String n) {    // method to set data  
        rollNo = r;  
        name = n;  
    }  
  
    void display() {                  // method to display data  
        System.out.println("RollNo: " + rollNo);  
        System.out.println("Name: " + name);  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        Student s1 = new Student();    // object creation  
        s1.setData(101, "Amit");        // calling method  
        s1.display();                  // calling method  
    }  
}
```

SAMPLE OUTPUT

```
RollNo: 101  
Name: Amit
```



IMPORTANT

- Without creating an object, non-static methods cannot be called.
- A class is a template, objects are real entities.

ABSTRACT METHODS AND CLASSES

KEY POINTS

- ★ Abstract method has no body; it is only declared.
- ★ Abstract class may contain abstract methods as well as concrete methods.
- ★ An abstract class cannot be instantiated.
- ★ A class that has one or more abstract methods must be declared abstract.
- ★ It provides a common interface and forces subclasses to provide their specific implementation.

Abstract classes and methods are used to achieve **abstraction** in OOP.

They provide a blueprint for other classes.

An abstract method is declared **without implementation**, and the subclass (derived class) **must provide the implementation**.

1. ABSTRACT METHOD

- A method which is declared without implementation is called an abstract method.
- It is declared using the keyword **abstract**.
- It must be overridden (implemented) in the subclass.

Syntax :

```
abstract return_type methodName(parameters);
```

Example :

```
abstract class Shape {  
    abstract double area(); // abstract method  
}
```

2. ABSTRACT CLASS

- A class that is declared using the keyword **abstract** is called an abstract class.
- It may contain abstract methods (without body) as well as concrete methods (with body).
- It cannot be instantiated.
- It is meant to be inherited.

Syntax :

```
abstract class ClassName {  
    // abstract method(s)  
    // concrete method(s)  
}
```

Example :

```
abstract class Shape {  
    abstract double area(); // abstract method  
    void display(); // concrete method  
    System.out.println("This is a shape");  
}
```

3. EXAMPLE PROGRAM

Abstract Class

```
abstract class Shape {  
    String color;  
    Shape(String c) {  
        color = c;  
    }  
    abstract double area(); // abstract method  
    void displayColor() { // concrete method  
        System.out.println("Color: " + color);  
    }  
}
```

Subclass (providing implementation)

```
class Circle extends Shape {  
    double radius;  
    Circle(String c, double r) {  
        super(c);  
        radius = r;  
    }  
    double area() { // implementation  
        return 3.14 * radius * radius;  
    }  
}
```

Using the classes

```
class Test {  
    public static void main(String[] args) {  
        Shape s;  
        s = new Circle("Red", 5.0); // upcasting  
        System.out.println("Area: " + s.area()); // calls overridden method  
        s.displayColor(); // calls concrete method  
    }  
}
```

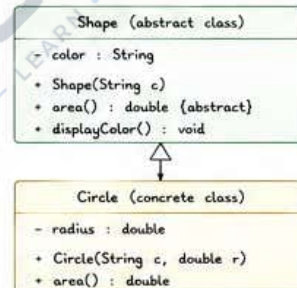
Output :

Area: 78.5
Color: Red

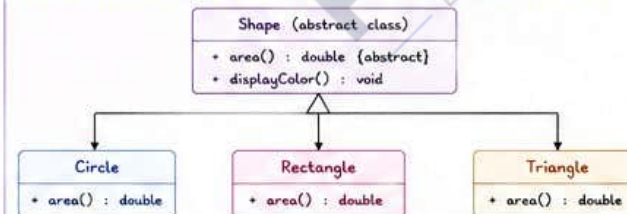
4. IMPORTANT RULES

- 1 If a class contains one or more abstract methods, the class must be declared abstract.
- 2 Abstract class cannot be instantiated.
- 3 Abstract method must be implemented in the subclass (otherwise subclass also must be abstract).
- 4 Abstract class can have constructors and static methods.
- 5 A reference of abstract class can point to the object of its subclass (upcasting).

5. CLASS DIAGRAM



6. ABSTRACT CLASS WITH MULTIPLE SUBCLASSES



Example :

```
Shape s;  
s = new Circle("Red", 5);  
s = new Rectangle("Blue", 4, 6);  
s = new Triangle("Green", 4, 6);  
// Polymorphism
```

7. SUMMARY TABLE

Feature	Abstract Method	Abstract Class	Concrete Class
Definition	Method without body	Class that may have abstract and concrete methods	Class with all methods having body
Keyword	abstract	abstract class	class
Body	No body	May have or not	Must have
Can be instantiated?	Not applicable	No	Yes
Must be overridden?	Yes (in subclass)	Not applicable	Not applicable

8. WHY USE ABSTRACT CLASSES?

- ✓ To provide a common interface.
- ✓ To achieve abstraction (hiding implementation details).
- ✓ To ensure that subclasses provide specific implementation.
- ✓ To support code reusability and polymorphism.

NOTE

- ★ An abstract class reference can hold a subclass object.
- ★ You cannot create an object of abstract class.
- ★ You can have a constructor in an abstract class.
- ★ Abstract methods are resolved at runtime (dynamic binding).

INTERFACES : INTRODUCTION & DEFINING INTERFACES

An interface is a reference type in Java, similar to a class, that can contain only constants, abstract methods, default methods, static methods and nested types.

It defines a contract (a set of rules) that implementing classes must follow.

KEY POINTS

- ★ Interfaces achieve 100% abstraction.
- ★ Used to achieve multiple inheritance (of type).
- ★ All methods in interface are public and abstract by default.
- ★ All fields in interface are public, static and final by default.
- ★ A class implements an interface using the keyword implements.

1. INTERFACES : INTRODUCTION

- An interface specifies what a class must do, not how.
- It is a blueprint for a class.
- It contains abstract methods (without body) and constants.
- Variables in interface are implicitly public, static and final.
- Methods in interface are implicitly public and abstract.
- A class implements an interface using the keyword **implements** and provides the body of methods.
- An interface can extend multiple interfaces.
- Interfaces are used to achieve loose coupling, abstraction and multiple inheritance.

Syntax

```
interface InterfaceName {  
    // constants  
    // abstract methods  
}
```

2. DEFINING INTERFACES

Syntax :

```
interface InterfaceName {  
    // constant declarations  
    // abstract method declarations  
}
```

Rules while defining interface :

1. Use the keyword interface.
2. The body contains only method declarations and constants.
3. Methods are declared without body (abstract methods).
4. Variables are public, static and final by default.
5. Methods are public and abstract by default.

Example :

```
interface Drawable {  
    int MAX_SIZE = 100; // public static final  
    void draw(); // public abstract  
    void clear(); // public abstract  
}
```

3. IMPLEMENTING AN INTERFACE

A class uses an interface by implementing it.

Syntax :

```
class ClassName implements InterfaceName {  
    // body of methods  
}
```

Example :

```
interface Drawable {  
    int MAX_SIZE = 100;  
    void draw();  
    void clear();  
}  
  
class Circle implements Drawable {  
    public void draw() {  
        System.out.println("Drawing Circle");  
    }  
    public void clear() {  
        System.out.println("Clearing Circle");  
    }  
}
```

Using the implemented interface :

```
class Test {  
    public static void main(String[] args) {  
        Drawable d = new Circle(); // upcasting  
        d.draw();  
        d.clear();  
        System.out.println(Drawable.MAX_SIZE);  
    }  
}
```

Output

```
Drawing Circle  
Clearing Circle  
100
```

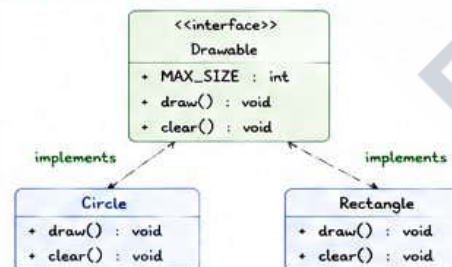
4. FEATURES OF INTERFACE

- Achieves abstraction.
- Supports multiple inheritance (a class can implement multiple interfaces).
- Provides a common contract for unrelated classes.
- Promotes loose coupling and better code reusability.
- All members are public by default.

5. INTERFACE MEMBERS (BY DEFAULT)

Member	Default Modifiers
Variables (Fields)	public static final
Methods	public abstract
Constructors	Not allowed
Blocks	Not allowed (except static block in Java 8 and above)
Nested Types	Allowed

6. CLASS DIAGRAM



7. INTERFACE VS ABSTRACT CLASS

Feature	Interface	Abstract Class
Purpose	100% abstraction	Partial abstraction
Methods	Abstract by default	Can have abstract and concrete methods
Variables	public static final	Can have any type of variables
Constructors	Not allowed	Allowed
Inheritance	Class can implement multiple interfaces	Class can extend only one abstract class
Keyword	interface	abstract class

8. INTERFACE EXTENDS INTERFACE

Syntax :

```
interface InterfaceA {  
    void methodA();  
}  
  
interface InterfaceB extends InterfaceA {  
    void methodB();  
}
```

Example :

```
interface A {  
    void showA();  
}  
  
interface B extends A {  
    void showB();  
}  
  
class Demo implements B {  
    public void showA() { System.out.println("A"); }  
    public void showB() { System.out.println("B"); }  
}
```

9. MULTIPLE INTERFACE IMPLEMENTATION

Example :

```
interface X {  
    void x();  
}  
  
interface Y {  
    void y();  
}  
  
class MyClass implements X, Y {  
    public void x() { System.out.println("X"); }  
    public void y() { System.out.println("Y"); }  
}
```

- ★ A class can implement any number of interfaces.

10. IMPORTANT NOTE

- ★ Interfaces cannot be instantiated.
- ★ Reference of interface can be created.
- ★ A class must implement all abstract methods of an interface.
- ★ Starting from Java 8, interfaces can have default and static methods.
- ★ Starting from Java 9, interfaces can have private methods.



EXTENDING INTERFACES

In Java, an interface can extend one or more interfaces using the keyword **extends**. The derived interface (sub-interface) inherits all the constants and abstract methods of the parent interfaces.

KEY POINTS

- ★ An interface can extend one or more interfaces.
- ★ It inherits all constants and abstract methods.
- ★ It can add its own abstract methods and constants.
- ★ It supports multiple inheritance of type.
- ★ The child interface is also abstract by default.

1. SYNTAX

Syntax :

```
interface ChildInterface extends ParentInterface1,
    ParentInterface2, ... {
    // new constants
    // new abstract methods
}
```

Example :

```
interface A {
    void methodA();
}
interface B {
    void methodB();
}
interface C extends A, B {    // C extends A and B
    void methodC();
}
```

2. HOW IT WORKS ?

- ✓ When interface C extends A and B, then C has all methods of A and B.
- ✓ The class that implements C must provide the implementation of all methods of A, B and C.
- ✓ C can also declare its own methods and constants.

4. EXAMPLE PROGRAM

Parent Interface 1 (A)

```
interface A {
    int X = 10;    // public static final
    void methodA();
}
```

Parent Interface 2 (B)

```
interface B {
    int Y = 20;    // public static final
    void methodB();
}
```

Child Interface C extending A and B

```
interface C extends A, B {
    int Z = 30;    // new constant
    void methodC();    // new method
}
```

Class implementing the child interface C

```
class MyClass implements C {
    public void methodA() {
        System.out.println("Method A");
    }
    public void methodB() {
        System.out.println("Method B");
    }
    public void methodC() {
        System.out.println("Method C");
    }
}
```

Using the class

```
class Test {
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.methodA();
        obj.methodB();
        obj.methodC();
        System.out.println("X = " + C.X);
        System.out.println("Y = " + C.Y);
        System.out.println("Z = " + C.Z);
    }
}
```

Output

```
Method A
Method B
Method C
X = 10
Y = 20
Z = 30
```

5. INTERFACE EXTENDS MULTIPLE INTERFACES

Example :

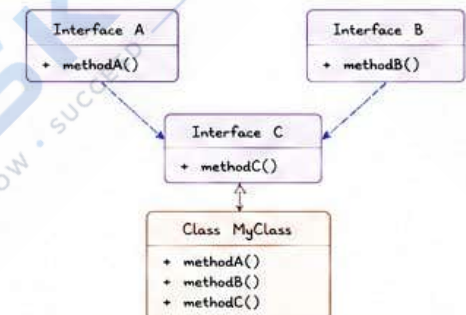
```
interface A { void a(); }
interface B { void b(); }
interface C { void c(); }

interface D extends A, B, C {
    void d();
}
```

Note :

A child interface can extend any number of interfaces, separated by commas.

6. CLASS DIAGRAM



3. FEATURES

- Supports multiple inheritance of type.
- Promotes code reusability.
- Helps achieve abstraction.
- All members are public by default.
- No constructor in interfaces.

7. RULES FOR EXTENDING INTERFACES

- ① An interface can extend one or more interfaces.
- ② The child interface inherits all constants and abstract methods.
- ③ The child interface can add its own constants and abstract methods.
- ④ All methods in the child interface are implicitly public and abstract.
- ⑤ A class implementing the child interface must implement all methods from all parent interfaces and the child interface.
- ⑥ Variables in interfaces are implicitly public, static and final.

8. KEY DIFFERENCE :

Class Extends Class	Interface Extends Interface
Single inheritance	Multiple inheritance
Gets implementation	Gets only method declarations
Can have constructors	Cannot have constructors
Members can be any type	Only public static final (variables) and public abstract (methods)

9. SUMMARY TABLE

Concept	Details
Keyword	extends
Extends	One or more interfaces
Inherits	All constants and abstract methods
Can add	Own constants and abstract methods
Implementation	By a class using implements keyword
Type of inheritance	Multiple inheritance of type



IMPORTANT NOTE

- An interface cannot be instantiated.
- An interface reference can be created.
- Starting from Java 8, interfaces can have default and static methods.
- Starting from Java 9, interfaces can have private methods.

IMPLEMENTING INTERFACES

A class implements an interface using the keyword **implements**.
The class must provide implementation for all **abstract methods** declared in the interface.

By implementing an interface, a class agrees to follow the **contract** (set of rules) defined by the interface.

KEY POINTS

- ★ A class implements an interface using the keyword **implements**.
- ★ The class must override all abstract methods of the interface.
- ★ Multiple interfaces can be implemented by a class.
- ★ The methods of interface are public by default.
- ★ A class can be both a subclass and an interface implementer.

1. SYNTAX

```
class ClassName implements InterfaceName {
    // implementation of abstract methods
}
```

Example :

```
interface Drawable {
    void draw();           // abstract method
    void clear();          // abstract method
}

class Circle implements Drawable {
    public void draw() {
        System.out.println("Drawing Circle");
    }
    public void clear() {
        System.out.println("Clearing Circle");
    }
}
```

2. HOW IT WORKS ?

- ✓ When a class implements an interface, it must provide body for all abstract methods.
- ✓ If a class does not implement all abstract methods, it must be declared abstract.
- ✓ The implementing class can add its own fields, methods and constructors.
- ✓ Reference of interface can be used to refer object of implementing class (Upcasting).

3. EXAMPLE PROGRAM

Interface

```
interface Shape {
    double area();    // abstract method
    void display();   // abstract method
}
```

Implementing Class

```
class Rectangle implements Shape {
    private double length, width;

    // constructor
    Rectangle(double l, double w) {
        length = l; width = w;
    }

    // implementation of abstract methods
    public double area() {
        return length * width;
    }
    public void display() {
        System.out.println("Rectangle");
    }
}
```

Using the Interface

```
class Test {
    public static void main(String[] args) {
        Shape s;           // interface reference
        s = new Rectangle(5, 4); // upcasting

        s.display();        // calls display()
        System.out.println("Area = " + s.area()); // calls area()
    }
}
```

Output :

```
Rectangle
Area = 20.0
```

4. IMPLEMENTING MULTIPLE INTERFACES

Example :

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class MyClass implements Printable, Showable {
    public void print() {
        System.out.println("Printing...");
    }
    public void show() {
        System.out.println("Showing...");
    }
}
```

5. PARTIAL IMPLEMENTATION USING ABSTRACT CLASS

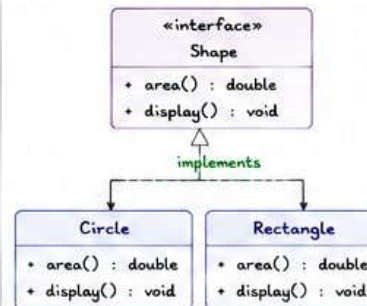
- ✓ If a class implements an interface but does not implement all methods, the class must be declared abstract.

Example :

```
interface Shape {
    double area();
    void display();
}

abstract class Base implements Shape {
    public void display() {
        System.out.println("This is a shape");
    }
    // area() left unimplemented
}
```

6. CLASS DIAGRAM



7. RULES

1. Class must implement all abstract methods of an interface.
2. Methods in interface are public and abstract.
3. Class can implement multiple interfaces separated by comma.
4. Variables in interface are public, static and final by default.
5. An interface cannot be instantiated, but its reference can be used.

8. INTERFACE VS ABSTRACT CLASS

	Interface	Abstract Class
Methods	All methods are abstract (by default)	Can have abstract and concrete methods
Variables	public static final (constants)	Can have instance variables
Constructors	Not allowed	Allowed
Multiple inheritance	Supported	Not supported
Keyword	implements	extends



IMPORTANT NOTE

- Interface defines what to do, not how to do.
- It provides total abstraction.
- Starting from Java 8, interfaces can have default and static methods.
- Starting from Java 9, interfaces can have private methods also.